Lezioni Algoritmi e Strutture Dati

Alberto Bernasconi

a.a 25/26

Indice

1 Efficienza Algoritmica e Problem Solving **2**

2 Complessità Computazionale **6**

3 Complessità Asintotica e Modelli di Calcolo **10**

4 Modelli di Calcolo e Semantica **13**

5 Valutazione Computazionale e Modello RAM **15**

6 Estensioni del Modello RAM e Teoria della Computabilità **19**

7 Linguaggi ad Alto Livello e Strutture Dati Elementari **24**

8 Strutture Dati: Dalla Memoria Statica all'Allocazione Dinamica **29**

9 Strutture Dati Dinamiche: Liste e Gestione della Memoria **33**

10 Strutture Dati Dinamiche e ad Accesso Ristretto **38**

11 Strutture Dati Dinamiche e Grafi: Implementazioni e Teoria degli Alberi **43**

12 Rappresentazione di Grafi e Teoria degli Alberi **47**

13 Proprietà degli Alberi e Loro Rappresentazioni **51**

14 Visita di Grafi e Alberi **56**

15 Eliminazione della Ricorsione e Introduzione all'Ordinamento **61**

Capitolo 1

Efficienza Algoritmica e Problem Solving

23 Febbraio 2026 **Il Problema della Celebrità**

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale: Problema della Celebrità**[1] L'assunzione matematica fondamentale stabilisce un'asimmetria perfetta nella conoscenza. Viene dimostrato per assurdo che in un gruppo la celebrità, se esiste, è unica.
- **Soluzione Naive**[2] Questo approccio richiede un numero di domande proporzionale a $O(n^2)$.
- **La soluzione:** Utilizzando la domanda atomica "X conosce Y?", si esplora l'asimmetria informativa. Se la risposta è affermativa, X viene scartato (non è una celebrità in quanto conosce qualcuno). Se la risposta è negativa, Y viene scartato (non è una celebrità in quanto non è conosciuto da X). Ogni domanda elimina un candidato. L'equazione di ricorrenza sul numero di domande necessarie per giungere a un singolo candidato è:

  $$C(n) = C(n-1) + 1$$

  La condizione iniziale prevede che, una volta isolato l'ultimo candidato, occorra verificarlo interrogandolo sulle restanti $n-1$ persone. Asintoticamente, la complessità dell'algoritmo ottimo decade a un costo lineare $\Theta(n)$. *Nota: Durante la lezione il calcolo esatto del numero di domande ($2n-3$ vs $2n-2$) viene ricalibrato, confermando comunque l'ordine di grandezza lineare.*

Guida di Studio Approfondita

Il Concetto Base: Immagina di dover scegliere un testimonial per una campagna pubblicitaria all'interno di una stanza con 100 persone. Se chiedi a ciascuno se conosce ogni singola altra

[1] **Problema della Celebrità:** Individuare, all'interno di una comunità di n persone, un individuo che non conosce nessuno ma che è conosciuto da tutti.

[2] **Soluzione Naive:** Ispezionare sequenzialmente ogni persona interrogandola su tutte le altre $n-1$ e verificando se le altre $n-1$ la conoscono.

persona, farai quasi 10.000 domande. Ma c'è un trucco logico: prendi due persone a caso, A e B, e chiedi ad A: "Conosci B?". Qualunque sia la risposta, puoi eliminare uno dei due! Se A dice "Sì", A non è la celebrità (perché la celebrità non conosce nessuno). Se A dice "No", B non è la celebrità (perché tutti devono conoscerla).

Analisi Passo-Passo:

1. **Fase di Eliminazione:** Prendi le n persone a coppie. A ogni domanda scarti una persona. Dopo $n-1$ domande, rimane un solo "sospettato".

2. **Fase di Verifica:** Il fatto che sia rimasto per ultimo non garantisce che sia la celebrità (potrebbe non esserci alcuna celebrità nel gruppo). Devi fare un ultimo giro di domande mirate per accertarti che lui non conosca gli altri e che gli altri conoscano lui.

Il "Perché" è importante: Questo problema dimostra concretamente l'importanza dell'efficienza algoritmica. Passare da una soluzione quadratica (Naive) a una lineare (Ottima) significa, in termini applicativi, risparmiare tempo e denaro. Se ogni domanda costa 10 centesimi e si analizza una città di 100.000 abitanti, l'algoritmo Naive costa 1.000 milioni di euro, mentre quello Ottimo solo 20.000 euro. L'hardware più potente non può compensare un algoritmo inefficiente.

Ripasso Visivo/Schematico:

Algoritmo	**Complessità**	**Costo Economico (100k input)**
Soluzione Naive	$O(n^2)$ (Quadratico)	Inaccettabile (Miliardi di euro)
Soluzione Ottima	$\Theta(n)$ (Lineare)	Gestibile (Decine di migliaia di euro)

23 Febbraio 2026 — Il Problema del Missionario Chirurgo

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale: Problema del Missionario Chirurgo**[3]

- **Limite Teorico**[4]

- **La soluzione:** L'algoritmo ottimo sfrutta l'impilamento delle risorse e il loro rovesciamento. Si infilano tutti gli n guanti in sequenza. Eseguita la prima operazione, il guanto più esterno (sporco all'esterno) viene sfilato e rovesciato (divenendo sporco all'interno e sterile all'esterno). Si procede ricorsivamente fino a 1 guanto. Successivamente, si reindossano i guanti rovesciati uno ad uno, ponendo a contatto superfici contaminate tra loro. L'equazione di ricorrenza associata è:

$$C(n) = C(n-1) + 2$$

La risoluzione della ricorrenza porta al limite teorico esatto di $2n-1$ operazioni.

[3]**Problema del Missionario Chirurgo:** Fissate n risorse finite (paia di guanti), determinare l'algoritmo che massimizza il numero di operazioni chirurgiche sicure effettuabili, dove per "sicura" si intende l'assenza di contatto tra superfici contaminate e superfici sterili.

[4]**Limite Teorico:** Un insieme di n paia di guanti ha $2n$ superfici totali (interne ed esterne). Poiché la mano del chirurgo contamina necessariamente 1 superficie all'inserimento, il numero massimo teorico di superfici sterili utilizzabili per il contatto con il paziente è $2n-1$.

Guida di Studio Approfondita

Il Concetto Base: Un chirurgo deve operare in condizioni di scarsità di risorse (es. in missione). Ha solo 10 paia di guanti, ma vuole fare più di 10 operazioni. Invece di usare un guanto e buttarlo, usa l'astrazione spaziale: le superfici del guanto sono due (interna ed esterna). Se isoliamo lo sporco, possiamo raddoppiare le operazioni.

Analisi Passo-Passo:

1. Il chirurgo infila tutte e 10 le paia di guanti una sopra l'altra (astrazione matematica: ignoriamo la perdita di sensibilità tattile).
2. Fa la 1° operazione. Il guanto più esterno si sporca fuori.
3. Lo sfila, lo *rovescia* (ora è sporco dentro, pulito fuori) e lo mette da parte.
4. Fa la 2° operazione col guanto sottostante (che era rimasto pulito).
5. Arrivato a operare con l'ultimo guanto rimasto (la 10° operazione), ricomincia la risalita: prende il primo guanto messo da parte (sporco dentro, pulito fuori) e lo indossa sopra quello che ha già. Lo sporco del nuovo guanto tocca lo sporco del guanto sottostante, ma l'esterno è pulito! Procedendo così, esegue altre 9 operazioni, arrivando a 19 totali.

Il "Perché" è importante: Questo problema introduce un concetto duale rispetto a quello della celebrità. Lì volevamo *minimizzare* l'uso delle risorse, qui le risorse sono bloccate (costanti) e vogliamo *massimizzare* l'output. Dimostra come il "pensiero algoritmico" debba prima identificare un limite teorico invalicabile ($2n - 1$ superfici), e poi trovare la sequenza di passi (l'algoritmo) per raggiungerlo.

Ripasso Visivo/Schematico:

Risorsa ($n = 10$)	**Risultato Ottenibile**
Soluzione Standard	10 interventi (uso lineare 1 a 1). La mano infetta n superfici interne sprecandole.
Soluzione Ottima	19 interventi. La mano infetta solo 1 superficie interna (del primo guanto), preservando le altre.

23 Febbraio 2026 — Il Problema dell'Equa Divisione

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale: Fair Division**[5] I tre vincoli inderogabili sono: 1. Impossibilità di coalizioni. 2. Se un individuo A riceve un sottoinsieme S_1 tale che Valore$(S_1) < R/K$, la responsabilità deve ricadere unicamente sulle scelte algoritmiche di A. 3. Assenza di invidia (Envy-Free): un individuo non può invidiare le quote altrui se ha ricevuto almeno una quota $\geq R/K$.
- **La soluzione:** Per $K = 2$, si applica il protocollo "Cut and Choose": un'entità partiziona in due sottoinsiemi, l'altra sceglie per prima. Per $K = 3$ (individui A, B, C): l'individuo A divide in tre insiemi S_1, S_2, S_3. L'individuo B individua l'insieme peggiore S_i (secondo B, Valore$(S_i) \leq R/3$). Se C concorda, l'insieme viene assegnato ad A, e si riduce a un

[5] **Fair Division:** Problema di partizione in cui K entità devono spartirsi un set di risorse di valore complessivo R.

problema per $K = 2$ tra B e C. Se C non concorda (secondo C, Valore(S_i) $> R/3$), l'insieme viene assegnato a C, riducendo a un problema per $K = 2$ tra A e B. *Nota: Se le risorse sono discrete (non liquidi continui), l'algoritmo può necessitare di meccanismi di compensazione finanziaria.*

Guida di Studio Approfondita

Il Concetto Base: Immagina che due fratelli litighino per dividersi un'eredità. Se uno fa le parti, rischia di fare parti disuguali. La soluzione logica perfetta è: "Il fratello A divide le risorse in due mucchi. Il fratello B sceglie per primo quale mucchio prendere". Il fratello A sarà costretto a fare le parti in modo chirurgicamente equo, altrimenti B gli porterà via la parte più grande. Nessuno dei due potrà lamentarsi (Envy-Free).

Analisi Passo-Passo: Se le risorse sono *continue* (es. un grosso boccale di birra da dividere tra 10 amici), il protocollo diventa dinamico:

1. La Persona 1 versa nel proprio bicchiere quella che reputa essere esattamente 1/10 della birra.
2. Passa il bicchiere alla Persona 2. Se la Persona 2 pensa che sia troppa, ne riversa un po' nel boccale principale. Se pensa che sia giusta, lo passa alla Persona 3.
3. Si continua così. Il bicchiere va all'ultima persona che ha riversato la birra (o alla Persona 1 se nessuno ha riversato).
4. Chi prende il bicchiere non può invidiare nessuno, perché ha dettato lui la quantità. Gli altri non possono lamentarsi perché hanno avuto la possibilità di ridurla e non l'hanno fatto.

Il "Perché" è importante: L'equa divisione mostra come gli algoritmi non servano solo a ordinare dati nei database, ma siano strumenti per risolvere conflitti applicando regole formali in grado di garantire stabilità ed equità nei sistemi distribuiti.

Ripasso Visivo/Schematico:

Parametro	**Regola Algoritmica**
$K = 2$	A divide, B sceglie (Cut and Choose).
$K = 3$	A divide, B indica il peggiore, C fa da arbitro discriminante.
Beni Continui	Assegnazione all'ultima entità che modifica al ribasso la quota.
Beni Discreti	Integrazione con moneta per compensare il valore degli indivisibili.

Capitolo 2

Complessità Computazionale

24 Febbraio 2026 **Algoritmi, Correttezza e Analisi della Complessità**

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale: Problema**[1] L'insieme degli ingressi per il problema deve coincidere esattamente con l'insieme degli ingressi che l'algoritmo è chiamato a processare.
- **Algoritmo**[2]
- **La soluzione:** L'algoritmo implementato per risolvere un problema si definisce *corretto* se e solo se, per ogni possibile istanza di input x appartenente al dominio, l'output dell'algoritmo $f_A(x)$ converge esattamente alla soluzione richiesta dal problema $f(x)$. Formalmente, il vincolo di correttezza richiede l'equivalenza funzionale assoluta:

$$\forall x \in D_I,\ f_A(x) = f(x)$$

Guida di Studio Approfondita

Il Concetto Base: Spesso si confondono "problema", "algoritmo" e "programma". Il *problema* è la richiesta matematica (es. "questo numero è primo?"). L'*algoritmo* è il metodo astratto per arrivare alla soluzione. Il *programma* è il codice concreto scritto in un linguaggio (es. Java) che gira su una macchina.

Analisi Passo-Passo:

1. Un algoritmo che va in loop infinito (es. un ciclo `while` senza condizione di uscita raggiungibile) calcola una funzione parziale: non termina e restituisce il "nulla" matematico ($\perp$).

[1] **Problema:** Una funzione che associa a un dominio di istanze di ingresso un codominio di soluzioni o risposte.
[2] **Algoritmo:** Una funzione calcolata che trasforma dati in ingresso in output. Può essere una funzione parziale qualora il calcolo non termini, generando un esito indefinito, indicato formalmente con $\perp$.

2. Un algoritmo *corretto* deve dare la risposta esatta per *tutti* gli input possibili, non solo per alcuni. Se la risposta è giusta nel 99% dei casi e sbaglia nell'1%, l'algoritmo non è considerato corretto per le nostre definizioni teoriche.

Il "Perché" è importante: Affidarsi ad algoritmi non corretti o probabilistici è accettabile in alcuni contesti molto limitati. Tuttavia, il calcolo deterministico e l'analisi algoritmica formale si fondano su procedure che garantiscono sempre il corretto funzionamento, per escludere a priori "oracoli" statistici quando serve certezza matematica.

Ripasso Visivo/Schematico:

Concetto	**Semantica**
Problema $f(x)$	L'obiettivo teorico e l'accoppiamento input-soluzione.
Algoritmo $f_A(x)$	Logica astratta. Parziale se include stati di non-terminazione ($\perp$).
Programma	L'algoritmo "tradotto" per un esecutore fisico (macchina).

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale: Casi di Complessità**[3]
- **Complessità in Media**[4]
- **La soluzione:** Se la probabilità di ogni istanza è incognita, si assume per semplicità una distribuzione uniforme (equiprobabilità). L'equazione di ricorrenza del costo medio si formula raggruppando le istanze con dimensione e costo equivalenti:

$$T_{medio}(n) = \sum_k k \cdot \left(\frac{\sigma(n,k)}{I_n} \right)$$

Dove I_n è il numero totale di istanze di dimensione n, e $\sigma(n,k)$ rappresenta la cardinalità dell'insieme delle istanze di dimensione n che richiedono esattamente un costo k.

Guida di Studio Approfondita

Il Concetto Base: Anche per input della stessa identica grandezza (es. un array di 1.000 elementi), l'algoritmo può faticare di più o di meno. Questa variabilità dipende dal "grado di disordine" interno ai dati. Si categorizza l'analisi in tre "personaggi":

- L'algoritmo fortunato (Gastone) che riceve i dati già perfetti: *Caso Migliore.*
- L'algoritmo sfortunato (Paperino) che riceve i dati nel formato più ostile possibile: *Caso Peggiore.*
- L'utente tipico che usa l'algoritmo tutti i giorni: *Caso Medio.*

Analisi Passo-Passo:

1. **Caso Peggiore:** È il limite di garanzia. Garantisce che, anche nello scenario più catastrofico, il sistema non supererà mai quel costo.

[3]**Casi di Complessità:** Valutazione delle risorse consumate (tempo/spazio) in funzione della dimensione n dell'istanza in ingresso, valutate in base al caso Peggiore (massimo costo), Migliore (minimo costo) e Medio.

[4]**Complessità in Media:** Il comportamento atteso dell'algoritmo. Calcolata come sommatoria pesata delle probabilità di occorrenza di ciascuna istanza moltiplicata per il suo costo computazionale.

2. **Caso Medio:** Estremamente complesso da calcolare matematicamente, perché richiede di conoscere o ipotizzare (con $1/I_n$) la distribuzione statistica esatta degli input.

3. **Caso Migliore:** Spesso banalizzato (es. controllare se un vettore è già ordinato impiega $O(n)$) ma fornisce scarsa utilità analitica pratica.

Il "Perché" è importante: La scelta del caso da analizzare dipende dal dominio di applicazione. Chi progetta un bancomat ottimizza per il *Caso Medio* in modo da servire bene la clientela tipica. Chi programma il reattore di una centrale nucleare deve usare l'analisi nel *Caso Peggiore*, perché un fallimento ai limiti estremi non è tollerabile.

Ripasso Visivo/Schematico:

Caso	**Risorsa Calcolata**	**Utilità Pratica**
Migliore (Gastone)	Minima assoluta	Praticamente nulla.
Medio (Utente tipico)	Statistica / Ponderata	Alta (sistemi transazionali e commerciali).
Peggiore (Paperino)	Massima garantita	Fondamentale (sistemi critici, safety-critical).

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale: Limiti dello Sviluppo Tecnologico**[5]
- **La soluzione:** Assumendo un incremento della velocità hardware pari a un fattore costante $c = 10$, l'equazione che calcola l'aumento della dimensione gestibile varia dipendendo rigorosamente dalla classe di complessità dell'algoritmo impiegato. Sia n la dimensione originaria e n' la nuova dimensione gestibile:

$$\begin{aligned} O(n) &\implies n' = 10 \cdot n \\ O(n^2) &\implies n' = \sqrt{10} \cdot n \\ O(n^3) &\implies n' = \sqrt[3]{10} \cdot n \\ O(2^n) &\implies n' = n + \log_2(10) \approx n + 3 \end{aligned}$$

Guida di Studio Approfondita

Il Concetto Base: Un errore comune è pensare che comprare un computer molto più veloce (es. 10 volte più potente) significhi poter risolvere automaticamente problemi 10 volte più grossi. Questo è vero solo se il nostro algoritmo ha un'efficienza eccellente. Se l'algoritmo è asintoticamente pessimo, il nuovo supercomputer non servirà quasi a nulla.

Analisi Passo-Passo:

1. Se usiamo un algoritmo **lineare** (es. scansione semplice), il problema scala perfettamente. Un PC 10 volte più veloce elaborerà un input 10 volte più grande.

2. Se usiamo un algoritmo **quadratico**, il vantaggio si comprime pesantemente: il fattore non è 10, ma la radice di 10 (≈ 3.16).

3. Se l'algoritmo è **esponenziale** (es. 2^n), è un disastro. Il nuovo PC non moltiplicherà la dimensione gestibile, ma ci aggiungerà solo una costante ridicola (circa +3). Passerai dall'ordinare 100 elementi a 103 elementi. Un guadagno totalmente irrilevante a fronte dell'investimento hardware.

[5] **Limiti dello Sviluppo Tecnologico:** La stima della dimensione massima del problema n che è possibile risolvere a fronte di un potenziamento esogeno dell'hardware dell'esecutore (es. processore c volte più veloce).

Il "Perché" è importante: Lo studio degli algoritmi dimostra inequivocabilmente che il *software* vince sempre sull'*hardware*. Progettare una soluzione algoritmica di classe polinomiale (invece che esponenziale) porta guadagni di rendimento incalcolabili che nessun progresso fisico dei circuiti di silicio potrà mai compensare da solo.

Ripasso Visivo/Schematico:

Complessità	**Incremento Dimensione (con PC 10x)**	**Valutazione**
$O(n)$ Lineare	$n' = 10 \times n$	Massima resa hardware.
$O(n^2)$ Quadratica	$n' = 3.16 \times n$	Resa ridotta ma tangibile.
$O(2^n)$ Esponenziale	$n' = n + 3.32$	Resa quasi nulla (Hardware vanificato).

Capitolo 3

Complessità Asintotica e Modelli di Calcolo

26 Febbraio 2026 **Notazioni Asintotiche per l'Analisi della Complessità**

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale:** La valutazione del tempo di calcolo e dello spazio di memoria di un algoritmo richiede di analizzare il comportamento della funzione di costo $T(n)$ al tendere della dimensione dell'input n all'infinito. Tale analisi permette di trascurare fattori costanti e termini di ordine inferiore.
- **O grande (O)**[1]
- **Omega grande (Ω)**[2]
- **Theta grande (Θ)**[3]
- L'analisi matematica più fine introduce anche la nozione di **Asintotico ($\sim$)**[4] e di **O piccolo (o)**[5]. *Nota: nelle registrazioni l'applicazione rigorosa dei limiti per "asintotico a" e "o piccolo" è introdotta in modo discorsivo, ma matematicamente si appoggia sulla convergenza dei rapporti.*
- **La soluzione:** L'impiego delle notazioni asintotiche consente di semplificare espressioni complesse di complessità isolando il termine dominante. Siano $f_1(n) = O(g_1(n))$ e $f_2(n) = O(g_2(n))$, la regola della somma stabilisce la riduzione analitica:

$$f_1(n) + f_2(n) = O(\max(g_1(n), g_2(n)))$$

---

[1] **O grande (O):** Limite superiore. $f(n) = O(g(n))$ se esistono costanti $c > 0$ e $n_0 \geq 0$ tali che $\forall n > n_0$, risulta $f(n) \leq c \cdot g(n)$. Indica che $f(n)$ è dominata da $g(n)$.

[2] **Omega grande (Ω):** Limite inferiore. $f(n) = \Omega(g(n))$ se esistono costanti $c > 0$ e $n_0 \geq 0$ tali che $\forall n > n_0$, risulta $f(n) \geq c \cdot g(n)$. Indica che $f(n)$ domina $g(n)$.

[3] **Theta grande (Θ):** Stesso ordine di grandezza. $f(n) = \Theta(g(n))$ se $f(n) = O(g(n))$ e contemporaneamente $f(n) = \Omega(g(n))$. La funzione $f(n)$ è limitata sia superiormente che inferiormente da $g(n)$ a meno di costanti moltiplicative.

[4] **Asintotico ($\sim$):** $\lim_{n\to\infty} f(n)/g(n) = 1$

[5] **O piccolo (o):** $\lim_{n\to\infty} f(n)/g(n) = 0$, che denota una funzione trascurabile rispetto a $g(n)$

Guida di Studio Approfondita e Semplificata

Il Concetto Base: Calcolare esattamente quante singole istruzioni esegue un programma è spesso impossibile o inutilmente complicato. Piuttosto che avere un calcolo esatto come "$3n^2 + 15n + 7$ istruzioni", ci interessa sapere "grosso modo" come si comporta il programma quando l'input diventa gigantesco. Le notazioni asintotiche funzionano come filtri che eliminano i dettagli insignificanti e ci lasciano solo il "cuore" della crescita del costo (es. n^2).

Analisi Passo-Passo:

1. **O Grande (Il Caso Peggiore):** Immaginiamo che O Grande sia un "tetto". Se un algoritmo è $O(n^2)$, significa che nel peggiore dei casi (Paperino) il tempo non crescerà mai più velocemente di una parabola. Garantisce un limite di sicurezza.

2. **Omega Grande (Il Limite Minimo):** Funziona all'opposto. È un "pavimento". Se l'algoritmo è $\Omega(n)$, significa che anche nel caso più fortunato possibile (Gastone), farà almeno una fatica lineare.

3. **Theta Grande (La Misura Esatta):** Se O Grande è il tetto e Omega Grande è il pavimento, Theta ci dice che tetto e pavimento coincidono (a meno di un coefficiente). L'algoritmo cresce esattamente come quella funzione.

Il "Perché" è importante: Semplificare i conti ci permette di confrontare gli algoritmi istantaneamente. Se un software è diviso in tre procedure che costano $O(n)$, $O(n^2)$ e $O(n^3)$, possiamo ignorare le prime due. La complessità totale sarà dettata dal "collo di bottiglia", ovvero $O(n^3)$.

Ripasso Visivo/Schematico:

Notazione	**Simbolo Matematico**	**Significato Pratico**
O Grande	$O(g(n))$	Cresce *al massimo* come $g(n)$. Limite superiore.
Omega Grande	$\Omega(g(n))$	Cresce *almeno* come $g(n)$. Limite inferiore.
Theta Grande	$\Theta(g(n))$	Cresce *esattamente* come $g(n)$. Stesso ordine.
Asintotico a	$\sim$	Stesso ordine e *stesso coefficiente* moltiplicativo.
O Piccolo	$o(g(n))$	È *trascurabile* rispetto a $g(n)$.

26 Febbraio 2026 — Modello di Calcolo: La Macchina RAM

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale:** Valutare l'efficienza assoluta di un algoritmo richiede di isolarlo dai fattori contingenti legati al livello tecnologico dell'esecutore fisico. È necessario definire una macchina astratta con regole formali invariabili.

- **Macchina RAM**[6]

- **Set di Istruzioni**[7]

[6] **Macchina RAM:** Random Access Machine. Un modello teorico provvisto di un nastro di ingresso (sola lettura), un nastro di uscita (sola scrittura), un Program Counter (Location Counter) e una memoria formata da una sequenza infinita di registri $R_0, R_1, \ldots, R_k$. L'ipotesi forte del modello è che ogni registro possa contenere un intero arbitrariamente grande.

[7] **Set di Istruzioni:** Un repertorio minimale simile all'assembly. Comprende operazioni di trasferimento (LOAD, STORE), operazioni aritmetiche sull'accumulatore R_0 (ADD, SUB, MULT, DIV), I/O (READ, WRITE) e salti (JUMP, JGTZ, JZERO, JBLANK, HALT). L'indirizzamento supportato è diretto, immediato (tramite il simbolo =) e indiretto (tramite asterisco $*$ per l'accesso referenziato).

- **La soluzione:** La macchina elabora un input in modo sequenziale partendo da uno stato iniziale standard. Il calcolo della complessità temporale sotto l'ipotesi del criterio di costo uniforme si ottiene mediante la somma unaria delle T istruzioni effettivamente eseguite fino allo stato di HALT. L'equazione per il tempo di calcolo uniforme è:

$$T_{RAM}(x) = \sum_{i=1}^{T} 1$$

Guida di Studio Approfondita e Semplificata

Il Concetto Base: I computer cambiano continuamente: quello che oggi è veloce, domani sarà obsoleto. Come possiamo dire se un *algoritmo* è buono in assoluto, senza farci ingannare dalla potenza del processore che stiamo usando? Creando un computer "finto" e immutabile: la Macchina RAM. È un modello spogliato di tutto (niente schermo, niente disco rigido) che fa solo operazioni essenziali pescando numeri da infiniti cassetti (i registri).

Analisi Passo-Passo:

1. **La Memoria Infinita:** La macchina RAM non dà mai errori di "Spazio Esaurito" (Out of Memory). Ha infiniti registri ($R_1, R_2, \dots$).

2. **L'Accumulatore (R_0):** È l'unico "tavolo da lavoro". Tutte le operazioni matematiche devono passare di qui. Se vuoi sommare due numeri, devi portare il primo nell'accumulatore, sommare il secondo all'accumulatore, e poi salvare il risultato altrove.

3. **I Nastri:** L'input si legge in sequenza, un numero alla volta, senza poter mai tornare indietro (come i vecchi registratori a cassetta). Stessa cosa per l'output.

Il "Perché" è importante: Usando questo modello base, possiamo contare riga per riga quante istruzioni assembly il nostro algoritmo impiega per risolvere un problema. Se su questo modello astratto contiamo 1.000 istruzioni, questo è un dato puro, slegato dai GHz del processore fisico reale. Ci permette di confrontare la teoria algoritmica oggettivamente.

Ripasso Visivo/Schematico:

Componente RAM Astratta	**Caratteristiche e Limiti**
Nastro di Ingresso	Sola lettura, scansione unidirezionale. I dati finiscono con un "Blank".
Nastro di Uscita	Sola scrittura, unidirezionale.
Memoria Dati	Infiniti registri. Capacità illimitata per singolo registro (intero arbitrario).
Accumulatore (R_0)	Registro privilegiato. Snodo obbligatorio per tutte le operazioni aritmetich
Programma	A sola lettura (nella RAM classica). Sequenza di istruzioni etichettate.

Capitolo 4

Modelli di Calcolo e Semantica

02 Marzo 2026 La Macchina RAM e la Semantica Operazionale

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale: Macchina RAM (Random Access Machine)**[1]
- L'assunzione forte del modello stabilisce che ogni singolo registro possa ospitare un intero relativo ($\mathbb{Z}$) arbitrariamente grande. Il registro R_0 riveste il ruolo privilegiato di accumulatore, snodo obbligatorio per le operazioni aritmetiche.
- **Stato della macchina (S)**[2]
- **La soluzione:** L'evoluzione della macchina è dettata dal ciclo Fetch-Decode-Execute su un set di istruzioni ristretto (es. `LOAD`, `STORE`, `ADD`, salti incondizionati e condizionati quali `JZERO` o `JBLANK`). Il calcolo del valore di un operando $V_S(OP)$ dipende unicamente dallo stato S e dalla modalità di indirizzamento:

$$V_S(OP) = \begin{cases} i & \text{se indirizzamento immediato } (= i) \\ S(i) & \text{se indirizzamento diretto } (i) \\ S(S(i)) & \text{se indirizzamento indiretto } (*i) \\ \perp & \text{in caso di configurazione indefinita/errore} \end{cases}$$

  La computazione si intende conclusa producendo un output valido qualora l'esecuzione raggiunga l'istruzione `HALT`. Se la macchina entra in uno stato non ammesso (es. indice di registro negativo, $\perp$) si ha un errore di esecuzione, mentre in assenza di terminazione la funzione parziale calcolata dal programma sul dato input diverge ed è matematicamente indefinita.

[1] **Macchina RAM (Random Access Machine):** Un modello di esecutore astratto ideato per valutare la complessità computazionale. È costituito da una memoria a sola lettura per il programma, un'unità centrale con un Location Counter (LC), nastri di I/O sequenziali e unidirezionali, e una memoria dati formata da infiniti registri numerati ($R_0, R_1, \ldots$).

[2] **Stato della macchina (S):** Una funzione S che definisce il contenuto di ogni singola componente hardware in un dato istante. Lo stato iniziale S_0 fissa tutti i registri a zero, l'indice di lettura/scrittura dei nastri sulla prima cella e il Location Counter a 1.

Guida di Studio Approfondita e Semplificata

Il Concetto Base: Per giudicare un algoritmo in modo universale, non possiamo basarci sulla potenza del computer di casa, perché un hardware obsoleto lo farebbe sembrare inefficiente. Dobbiamo astrarre. La Macchina RAM è un "computer finto" scarnificato: niente hard disk o schede video, ma solo una scrivania su cui poggiare i calcoli correnti (l'accumulatore R_0) e un magazzino con infiniti cassetti numerati ($R_1, R_2 \dots$). I dati arrivano su un nastro come in una vecchia cassetta: li leggi uno a uno, e quando finiscono trovi un simbolo speciale di "vuoto" (`blank`).

Analisi Passo-Passo:

1. **Gestione degli Operandi:** La macchina può recuperare l'ingrediente per il suo calcolo in tre modi. Se il codice dice `=10`, l'ingrediente è esattamente il numero 10 (immediato, zero accessi). Se dice `10`, bisogna aprire il cassetto 10 e prendere ciò che c'è dentro (diretto, 1 accesso). Se dice `*10`, apro il cassetto 10, trovo un numero (es. 50), poi vado ad aprire il cassetto 50 per prendere il dato vero (indiretto, 2 accessi).

2. **Evoluzione della Memoria ("Fotografia" dello Stato):** Ogni istruzione eseguita crea un nuovo stato. Eseguire `LOAD 5` aggiorna solo il valore registrato su R_0, scattando una nuova foto dell'hardware. Tutti gli altri infiniti cassetti e le testine rimangono identici allo scatto precedente.

3. **Cicli e Fine Dati:** Visto che il nastro input si muove solo a destra, il programma non sa quanti elementi ci sono. L'istruzione `JBLANK` funge da sensore: appena legge il vuoto, fa saltare il programma all'istruzione di conclusione (es. stampare la somma finale e dare `HALT`).

Il "Perché" è importante: Senza questo costrutto formale e le sue definizioni matematiche (stato S, funzione V_S), non potremmo legare una semantica operazionale (come agisce fisicamente l'esecutore) con una nozione teorica astratta. L'indefinito matematico ($\perp$) in questo modello cattura l'assenza pratica di terminazione (i loop infiniti) e i crash (gli errori a runtime, come cercare un cassetto col numero -18).

Ripasso Visivo/Schematico:

Indirizzamento	**Sintassi RAM**	**Azione Logica in Memoria (Stato S)**
Immediato	`=i`	Prende direttamente la costante i. Nessun accesso alla memoria dati.
Diretto	`i`	Interroga una volta la memoria. Restituisce il contenuto $S(i)$.
Indiretto	`*i`	Interroga due volte la memoria. Restituisce il contenuto del registro puntato: $S(S(i))$.

Capitolo 5

Valutazione Computazionale e Modello RAM

03 Marzo 2026 Macchina RAM: Lettura Dati Dinamica e Indirizzamento Indiretto

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale:** Trasferire una sequenza di dati di dimensione n ignota a priori dal nastro di ingresso alla memoria di lavoro della Macchina RAM. La condizione di terminazione è definita dall'incontro del simbolo convenzionale di `blank` sul nastro.
- **Indirizzamento Indiretto**[1]
- **La soluzione:** Poiché non è possibile codificare a priori gli indici dei registri di destinazione (es. `READ 1`, `READ 2`, ...), l'algoritmo fa uso dell'accumulatore (`R0`) come contatore degli indici. Inizializzato a 1, l'algoritmo esegue un ciclo di lettura:
  1. Se c'è `blank`, salta all'uscita (`JBLANK`).
  2. Esegue `READ *0`: legge il dato dal nastro e lo memorizza nel registro il cui indice è attualmente salvato in `R0`.
  3. Incrementa l'accumulatore: `ADD =1`.
  4. Ripete il ciclo (`JUMP`).

  Così facendo, il programma popola dinamicamente i registri `R1`, `R2`, ..., `Rn` impiegando uno spazio asintotico di $n + 1$ registri.

Guida di Studio Approfondita e Semplificata

Il Concetto Base: Immaginiamo di dover svuotare un camion pieno di pacchi (il nastro di input) e metterli in magazzino in scatole numerate consecutivamente. Il problema è che non sappiamo quanti pacchi ci siano finché non finiscono. Se scrivessimo il codice "a mano" per ogni

[1] **Indirizzamento Indiretto:** Modalità di accesso in cui il valore operando specifica l'indice di un registro che a sua volta contiene l'indirizzo finale del dato desiderato. Nel linguaggio RAM si denota con l'asterisco (es. `*0`).

pacco, a un certo punto finiremmo le istruzioni. La soluzione è usare un foglietto (l'accumulatore R_0) su cui scriviamo "Scatola 1". Leggiamo un pacco e, guardando il foglietto (*indirizzamento indiretto*), lo mettiamo nella Scatola 1. Poi cancelliamo "1", scriviamo "2", e ripetiamo finché i pacchi non finiscono.

Analisi Passo-Passo:

1. L'accumulatore funge da "puntatore": tiene traccia dello spazio di memoria attualmente libero e disponibile.

2. L'istruzione `READ *0` è il cuore dell'algoritmo: senza l'asterisco, sovrascriveremmo sempre e solo il registro zero. Con l'asterisco, il processore sa che deve prima guardare in R_0 per ottenere la destinazione vera e propria.

Il "Perché" è importante: Questo approccio dimostra la potenza dell'indirizzamento indiretto: è il principio base alla fondazione dei puntatori e dell'allocazione dinamica della memoria nei linguaggi di alto livello (come array dinamici o liste).

Ripasso Visivo/Schematico:

Istruzione	Modalità	Effetto Logico
`=1`	Immediato	L'operando è il numero 1 in sé.
`1`	Diretto	L'operando è il contenuto del cassetto n° 1.
`*0`	Indiretto	Guarda nel cassetto 0, leggi il numero X, vai al cassetto n° X.

03 Marzo 2026 Il Criterio di Costo Uniforme e Analisi dei Casi

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale:** Misurare oggettivamente le risorse di Spazio e Tempo computazionale consumate da un algoritmo, svincolandole dai tempi di orologio legati alle frequenze fisiche dell'hardware (es. i GHz del processore).

- **Criterio di Costo Uniforme**[2]

- **La soluzione:** Fissato il criterio uniforme, la Complessità Temporale coincide col numero esatto di istruzioni eseguite. L'analisi richiede di valutare la funzione di costo $T(n)$ differenziandola tra Caso Migliore e Caso Peggiore. Analizzando ad esempio l'algoritmo di ricerca del Massimo su n elementi:

  - **Caso Migliore:** Input ordinato decrescente (il massimo è il primo elemento). Il ciclo salta la fase di aggiornamento del registro del massimo. $T(n) = 5(n-1) + \text{costanti}$.
  - **Caso Peggiore:** Input ordinato crescente. Ad ogni passo viene scoperto un nuovo massimo, forzando l'esecuzione delle istruzioni di aggiornamento. $T(n) = 8(n-1) + \text{costanti}$.

  In entrambi i casi, l'algoritmo appartiene alla classe asintotica lineare $\Theta(n)$.

[2]**Criterio di Costo Uniforme:** Modello di analisi in cui l'esecuzione di ogni singola istruzione RAM e l'occupazione di un singolo registro di memoria richiedono esattamente un'unità di costo ($O(1)$), indipendentemente dalla grandezza numerica dei dati elaborati.

Guida di Studio Approfondita e Semplificata

Il Concetto Base: Un vecchio computer degli anni '70 per fare una somma impiegava secondi; un computer moderno impiega miliardesimi di secondo. Come facciamo a dire se un *algoritmo* è lento o veloce se i computer cambiano? Semplice: non misuriamo i secondi, ma contiamo quante singole istruzioni (o "righe di codice") l'algoritmo esegue. Questa è l'essenza del costo uniforme.

Analisi Passo-Passo:

1. **Spazio Finito $\neq$ Tempo Finito:** Un'osservazione fondamentale fatta a lezione è che usare una memoria finita (es. solo 3 registri) non garantisce che l'algoritmo finirà. Si può costruire un ciclo infinito (`while true`) che usa un solo registro.

2. **Scarto prestazionale interno:** Anche contando le istruzioni, un array di 100 elementi può richiedere 500 operazioni o 800 operazioni a seconda di "come" sono disposti i dati. Il Caso Peggiore (800) diventa la nostra "garanzia": sappiamo che l'algoritmo non impiegherà mai più di così.

Il "Perché" è importante: Valutare il tempo come "numero di passi astratti" ci permette di confrontare la bontà matematica di due algoritmi in assoluto. Il Criterio Uniforme è quello che si usa nel 99% delle analisi software quotidiane, perché in genere la grandezza dei dati che mettiamo nei registri (es. interi a 32 o 64 bit) non causa scarti di tempo visibili.

Ripasso Visivo/Schematico:

Parametro (Es. Ricerca Max)	**Caso Migliore**	**Caso Peggiore**
Disposizione Input	Ordinamento Decrescente	Ordinamento Crescente
Passi nel Loop	5 istruzioni	8 istruzioni (aggiornamenti extra)
Complessità Asintotica	$\Theta(n)$	$\Theta(n)$

03 Marzo 2026 Criterio di Costo Logaritmico: Limiti e Paradosso dell'Esponenziale

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale:** L'assunzione del Criterio di Costo Uniforme (ogni dato occupa una cella e richiede tempo 1 per l'elaborazione) fallisce se la dimensione fisica del singolo operando supera l'effettiva capacità architetturale, trasformando un'operazione atomica in una routine software costosa.

- **Criterio di Costo Logaritmico**[3]

- **La soluzione:** Per evidenziare questa asimmetria, si considera un algoritmo che calcola il valore $y = 3^{2^n}$ tramite n quadrature successive ($y = y \times y$).

  - **In Costo Uniforme:** Il loop esegue n iterazioni con un numero costante di passi. Tempo $T(n) = \Theta(n)$.

[3] **Criterio di Costo Logaritmico:** Modello in cui il costo di memorizzazione ed elaborazione di un intero k è dato dalla sua lunghezza in bit, misurabile come $\lfloor \log_2 |k| \rfloor + 1$. Il costo di un'istruzione diviene dipendente dalla somma delle lunghezze dei suoi operandi.

- **In Costo Logaritmico:** All'iterazione k, la variabile y contiene il valore 3^{2^k}. Il numero di bit necessari è proporzionale a $\log_2(3^{2^k}) = 2^k \cdot \log_2(3)$. Il costo dell'operazione di moltiplicazione atomica è dominato dalla dimensione dell'operando: $O(2^k)$. La sommatoria su n passi genera un costo totale $T_{log}(n) = \sum_{k=1}^{n} 2^k = \Theta(2^n)$.

Il criterio logaritmico ristabilisce la corretta aderenza dell'analisi all'implementazione fisica.

Guida di Studio Approfondita e Semplificata

Il Concetto Base: Immagina di dover sommare due numeri. Se devi fare $5 + 3$, ci metti un istante. Se devi fare (8374928374) + (2394872938), ci metti molto di più perché devi incollare insieme molte cifre. Il Criterio Uniforme ignora questo fatto, dicendo che "una somma costa 1" a prescindere dai numeri. Finché restiamo sotto i limiti dei bit di un computer (es. 64-bit), va bene. Ma se generiamo numeri immensi (come $3^{2^{100}}$), il costo uniforme mente spudoratamente. Il Criterio Logaritmico fa invece pagare una "tassa" basata su quanto sono lunghi i numeri in bit.

Analisi Passo-Passo (Il paradosso del quadrato):

1. Immaginiamo l'algoritmo che fa il quadrato di un numero di continuo per n volte: $y = y \times y$.
2. Se $n = 10$, il loop gira solo 10 volte. Per il costo uniforme, l'algoritmo è fulmineo (costo 10).
3. Ma al decimo giro, il numero calcolato è 3^{1024}, un numero astronomico che richiede centinaia di bit per essere memorizzato.
4. Con il Costo Logaritmico, non contiamo "1 iterazione = 1 passo", ma contiamo la grandezza del numero al passo k. Poiché la grandezza raddoppia ad ogni giro (2^k), la somma finale non è lineare, ma esponenziale!

Il "Perché" è importante: Rivela che un'astrazione matematica (memoria e capacità infinita dei registri RAM) se applicata ciecamente può portare a disastri ingegneristici. Un algoritmo giudicato lineare ed eccellente sulla carta, si rivela bloccante ed esponenziale in fase di calcolo reale. Per questi algoritmi è obbligatorio usare il Costo Logaritmico.

Ripasso Visivo/Schematico:

Calcolo di 3^{2^n}	**Valutazione Criterio Uniforme**	**Valutazione Criterio Logaritmico**
Iterazioni Totali	n	n
Costo Moltiplicazione	1 unità (costante)	$\propto 2^k$ (cresce col numero di bit)
Complessità Finale	$\Theta(n)$ **(Lineare)**	$\Theta(2^n)$ **(Esponenziale)**
Esito Pratico	Informazione errata e inapplicabile	Esatta modellazione del calcolo reale

Capitolo 6

Estensioni del Modello RAM e Teoria della Computabilità

05 Marzo 2026 Criterio di Costo Logaritmico: Analisi dello Spazio e del Tempo

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale: Complessità in Spazio Logaritmico**[1]
- Il criterio uniforme fallisce quando i dati elaborati non possono essere contenuti entro la dimensione costante di un registro fisico.
- **La soluzione:** La formula matematica per la complessità in spazio logaritmico è:

$$S_{log}(n) = \max_{i \geq 0} \left(\sum_j \lfloor \log_2 |S_i(j)| \rfloor + 1 \right)$$

  Dove $S_i(j)$ rappresenta il contenuto del registro j nello stato i.
- Applicando questo criterio all'algoritmo di ricerca del **Massimo** tra n valori limitati da una costante K, lo spazio occupato asintoticamente è $O(\log K)$ (usiamo l'O grande perché non sappiamo se K compare effettivamente, ma è il limite superiore). Il tempo di calcolo diventa $\Theta(n)$ iterazioni per un costo massimo di $O(\log K)$ a istruzione, risultando in un tempo totale $O(n \log K)$.
- Applicando il criterio alla **Somma** di un vettore di n elementi tutti di valore n, lo spazio misurerà l'occupazione della somma finale $n \times n = n^2$. Poiché $\log_2(n^2) = 2 \log_2 n$, lo spazio è $\Theta(\log n)$. Il tempo calcola n iterazioni, ciascuna dominante per un costo di memorizzazione $\log n$, portando a $\Theta(n \log n)$.

[1] **Complessità in Spazio Logaritmico:** Il massimo spazio logaritmico occupato tra tutti gli stati della computazione. Nello stato i, si somma la lunghezza in bit del contenuto di ciascun registro utilizzato.

Guida di Studio Approfondita

Il Concetto Base: Fino ad ora abbiamo valutato lo spazio contando "quanti cassetti" (registri) un algoritmo apre. Nel criterio logaritmico, andiamo a misurare "quanto è grande l'oggetto" dentro ciascun cassetto misurandolo in bit. Se stiamo calcolando una somma che diventa enorme (come n^2), non basta dire che occupa 1 registro. Occupa uno spazio proporzionale alla lunghezza in bit del numero, ovvero il suo logaritmo in base 2.

Analisi Passo-Passo:

1. **Valutazione Spaziale:** L'algoritmo attraversa vari stati nel tempo $(S_0, S_1, S_2 \dots)$. Per ogni stato si fotografano tutti i registri usati e si sommano i bit dei numeri contenuti. Il picco massimo raggiunto in questa evoluzione decreta lo spazio logaritmico finale.

2. **Valutazione Temporale:** Invece di contare ogni riga di codice come "1", si moltiplica il numero di volte che l'istruzione viene eseguita per la lunghezza in bit dei numeri che manipola in quel momento.

Il "Perché" è importante: Ignorare la crescita dei dati durante l'esecuzione porta ad analisi errate. Affermare che la somma di n numeri tutti uguali a n costi $\Theta(n)$ (criterio uniforme) è ingannevole su macchine reali, perché i registri fisici andranno in overflow. L'analisi logaritmica ristabilisce la realtà fisica indicando un costo $O(n \log n)$.

Ripasso Visivo/Schematico:

Algoritmo (Input n)	**Costo Uniforme**	**Costo Logaritmico**
Ricerca Max ($\leq K$)	Tempo: $\Theta(n)$, Spazio: $\Theta(1)$	Tempo: $O(n \log K)$, Spazio: $O(\log K)$
Somma (Tutti $= n$)	Tempo: $\Theta(n)$, Spazio: $\Theta(1)$	Tempo: $\Theta(n \log n)$, Spazio: $\Theta(\log n)$

05 Marzo 2026 La Macchina RASP e l'Equivalenza dei Modelli

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale: Macchina RASP (Random Access Stored Program)**[2]
- **La soluzione:** Per dimostrare l'universalità della RAM, occorre provare che la macchina RASP, pur essendo priva di indirizzamento indiretto, possiede capacità equivalenti mediante l'uso del *codice automodificante*.
- **Teorema di Equivalenza RAM-RASP**[3]
- La prova è costruttiva: il compilatore converte un'istruzione RAM in una o più istruzioni RASP. Si partiziona la memoria shiftando i registri dati di un fattore $R = 12 \cdot (\text{numero istruzioni}) + 1$.
- L'indirizzamento indiretto RAM (es. `MULT *K`) viene simulato su RASP in 6 passi:

[2] **Macchina RASP (Random Access Stored Program):** Un modello di esecutore in cui istruzioni e dati risiedono nella medesima memoria modificabile. Le istruzioni occupano due registri contigui (Codice Operativo e Operando). Viene rimosso l'indirizzamento indiretto.

[3] **Teorema di Equivalenza RAM-RASP:** Per ogni programma RAM esiste un programma RASP equivalente. Il tempo di calcolo su RASP rispetta il limite: $T_{RASP}(x) \leq 6 \cdot T_{RAM}(x)$.

1. Salvataggio dell'accumulatore ($R_0 \to R_1$).
2. Calcolo del primo puntatore caricando in R_0 il contenuto del registro RASP $R + K$.
3. Somma dell'offset R ($R_0 = R_0 + R$). R_0 contiene ora l'indice fisico RASP del dato.
4. Sovrascrittura: si salva il valore calcolato in R_0 nel campo operando della sesta istruzione del blocco simulato.
5. Ripristino dell'accumulatore ($R_1 \to R_0$).
6. Esecuzione della sesta istruzione (la cui destinazione è stata calcolata dinamicamente).

- *Nota sulle registrazioni: Viene enunciato, ma non dimostrato formalmente, anche il teorema inverso (da RASP a RAM), specificando che la dimostrazione passa attraverso la costruzione di un interprete.*

Guida di Studio Approfondita

Il Concetto Base: La RAM ha un "difetto" di astrazione: il codice del programma è separato dai dati e immutabile (architettura Harvard estremizzata). I computer reali (architettura Von Neumann) hanno programma e dati insieme. La Macchina RASP corregge questo, fondendo le memorie. Per bilanciare la novità, le togliamo l'indirizzamento indiretto (i puntatori). Ci chiediamo: può questa macchina "menomata" fare le stesse cose della RAM? Sì, perché potendo scrivere nella propria memoria, il programma può "riscrivere se stesso" in tempo reale.

Analisi Passo-Passo (Il trucco della simulazione RASP): Se nella RAM vogliamo moltiplicare per il contenuto della cella indicata da K (`*K`), la RASP non sa farlo nativamente. Allora esegue una piccola procedura software:

1. "Mette in pausa" i calcoli correnti salvandoli di lato.
2. Va a leggere dove punta K e fa i calcoli per capire in che cella fisica si trovi il vero dato.
3. Prende l'istruzione di moltiplicazione generica scritta poche righe sotto e ne *sovrascrive* l'indirizzo vuoto col numero della cella appena calcolata.
4. Ripristina i calcoli e lascia che la macchina esegua l'istruzione appena alterata.

Il "Perché" è importante: Questo teorema dimostra in modo formale un pilastro dell'informatica: togliere una funzione hardware avanzata (i puntatori) non diminuisce la potenza matematica del computer, purché si possa compensare via software (pagando un piccolo costo in tempo, qui un fattore 6×).

Ripasso Visivo/Schematico:

Caratteristica	**Macchina RAM**	**Macchina RASP**
Memoria Dati e Codice	Separate (Programma in Sola Lettura)	Unificate (Il codice può alterare se stesso)
Indirizzamento Indiretto	Supportato Nativamente (*)	Non Supportato
Ingombro Istruzioni	1 unità logica	2 Registri (Codice + Operando)
Velocità Esecuzione	Riferimento Base (1×)	Peggioramento costante ($\leq$ 6×)

05 Marzo 2026 Classi di Complessità e Tesi di Church-Turing

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale:** Valutare l'equivalenza funzionale tra diversi linguaggi di programmazione o modelli astratti (RAM, RASP, Pascal, Java, Lisp, Macchina di Turing).
- **Tesi di Church-Turing**[4] Tutti i modelli di esecutore ragionevoli (RAM, RASP, ecc.) sono funzionalmente equivalenti.
- **La soluzione:** Il passaggio dalla calcolabilità astratta alla *calcolabilità effettiva* richiede l'analisi delle risorse necessarie. Fissando una specifica funzione limite $f(n)$ (es. tempo $O(n^2)$), l'equivalenza tra architetture cade a causa dei costi di simulazione. Tuttavia, emerge una nuova forma di invarianza.
- **Tesi di Church-Turing Estesa**[5]
- Se un problema è risolvibile su RAM con complessità $O(n^k)$, la sua simulazione su RASP richiederà un tempo $O(n^{k'})$, preservando la natura polinomiale della soluzione, seppur con un aumento del grado del polinomio.

Guida di Studio Approfondita

Il Concetto Base: Ci sono due grandi livelli di pensiero nell'informatica teorica. Il primo: "Si può risolvere questo problema con un computer?". La risposta è sì per la RAM, per il tuo Mac e per un supercomputer, ed è no per tutti sulla stessa identica classe di problemi insolubili (Tesi di Church-Turing). Il secondo livello è: "Si può risolvere *in tempi utili*?". Se scopri che un algoritmo è veloce (polinomiale) su un modello astratto, puoi stare certo che sarà polinomiale e usabile su qualsiasi computer reale (Tesi Estesa).

Analisi Passo-Passo:

1. **Calcolabilità Assoluta:** Due modelli (es. Java vs RASP) sanno risolvere esattamente gli stessi problemi. Uno non è logicamente "più intelligente" dell'altro.
2. **Perdita di Precisione Asoluta ($O(n^2)$):** Se scrivo un programma Java che fa un compito in tempo $O(n^2)$ e lo devo tradurre per una macchina povera come la RASP, le istruzioni base che deve simulare faranno salire il tempo a $O(n^3)$ o $O(n^4)$. La complessità specifica si perde.
3. **Salvezza Polinomiale:** Nonostante il rallentamento, la traduzione non trasformerà mai un problema polinomiale (efficiente) in uno esponenziale (impossibile).

Il "Perché" è importante: Questo ci autorizza a progettare e analizzare gli algoritmi usando strumenti teorici "finti" o pseudo-linguaggi ad alto livello. Non dobbiamo preoccuparci se il processore Intel ci mette 3 cicli di clock e l'ARM ce ne mette 5. Se dimostriamo matematicamente che l'algoritmo è polinomiale in teoria, garantiamo all'ingegnere che l'applicazione sarà scalabile nella pratica su qualsiasi architettura.

Ripasso Visivo/Schematico:

[4]**Tesi di Church-Turing:** Esiste un'unica classe universale di funzioni calcolabili per via automatica, coincidente con la classe delle *funzioni ricorsive parziali*.

[5]**Tesi di Church-Turing Estesa:** La classe di funzioni calcolabili in tempo polinomiale è invariante rispetto al modello di calcolo scelto, purché questo sia deterministico e fisicamente ragionevole.

Ambito di Indagine	**Domanda Principale**	**Invarianza tra Modelli**
Calcolabilità Teorica	Esiste un algoritmo che termina? (Sì/No)	Assoluta (Funzioni Ricorsive Parziali)
Complessità Specifica	Costa esattamente $O(n^2)$?	Falsa (Il grado del polinomio cambia per via delle simulazioni)
Calcolabilità Effettiva (Classe Polinomiale)	È calcolabile in tempo $O(n^k)$ per un qualche k?	Assoluta (Tesi di Church Estesa)

Capitolo 7

Linguaggi ad Alto Livello e Strutture Dati Elementari

09 Marzo 2026 Il Linguaggio AG, Variabili e Sottoprogrammi

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale: Linguaggio AG (Algol-like)**[1]
- **Variabile**[2]
- **La soluzione:** Per astrarre le computazioni si ricorre all'uso di sottoprogrammi. Si distinguono in *funzioni* (restituiscono un valore al chiamante tramite istruzione `return`) e *procedure* (non restituiscono valori, ma modificano lo stato della macchina). La comunicazione tra chiamante e chiamato avviene tramite il passaggio di parametri allocati nel Record di Attivazione (Stack):
  - **Passaggio per Valore (Copia):** Viene copiato il *Right-value* del parametro attuale. Il costo computazionale dipende dalla dimensione dell'oggetto.
  - **Passaggio per Riferimento (Indirizzo):** Viene copiato il *Left-value* del parametro attuale. Il costo computazionale è dominato da $\Theta(1)$ indipendentemente dalla dimensione dell'oggetto.
- Il costo dei costrutti di controllo si valuta asintoticamente. Ad esempio, per un selettore a due vie (`if P then C1 else C2`), il costo è limitato da:

$$T \leq T(P) + \max(T(C1), T(C2))$$

  Per un ciclo `for`, il costo è la sommatoria dei costi delle singole iterazioni, che possono dipendere dall'indice del ciclo:

$$T = \sum_{i=1}^{n} T(C_i)$$

[1] **Linguaggio AG (Algol-like):** Uno pseudolinguaggio procedurale di alto livello, svincolato da rigidità sintattiche e dichiarazioni di tipo esplicite, progettato per esprimere algoritmi in forma astratta superando i limiti di leggibilità del modello RAM.

[2] **Variabile:** Un alias per una zona di memoria. Possiede due semantiche in funzione della posizione in un'istruzione di assegnamento: *Left-value* (l'indirizzo di memoria, usato a sinistra dell'operatore di assegnamento) e *Right-value* (il contenuto o valore, usato a destra dell'operatore di assegnamento).

Guida di Studio Approfondita

Il Concetto Base: Perché abbandonare il linguaggio macchina RAM? Perché scrivere un programma complesso in assembly RAM richiede centinaia di righe incomprensibili. Il linguaggio AG ci permette di usare concetti naturali come `while`, `if`, e `procedure`. Quando usiamo una variabile (es. `X`), il computer deve sapere se ci interessa "dove si trova" (il suo indirizzo, il *Left-value*) o "cosa c'è dentro" (il suo valore, il *Right-value*). Se scriviamo `X := Y`, prendiamo il contenuto di `Y` e lo mettiamo nell'indirizzo di `X`.

Analisi Passo-Passo (Passaggio di Parametri): Immagina di avere una procedura `Scambia(X, Y)` che deve invertire i valori di due variabili.

1. Se usi il **Passaggio per Valore**: Stai dando alla procedura una fotocopia dei tuoi dati. La procedura scambia i valori sulle fotocopie. Quando la procedura finisce e butta via le fotocopie, i tuoi dati originali sono rimasti identici. La procedura è inutile.

2. Se usi il **Passaggio per Riferimento**: Stai dando alla procedura le "chiavi di casa" (l'indirizzo di memoria). La procedura entra a casa tua e sposta i mobili originali. Quando finisce, lo scambio è permanente.

Il "Perché" è importante: Scegliere come passare i parametri ha un impatto enorme sull'efficienza. Se passi una matrice da 1 milione di elementi "per valore", il processore deve copiare 1 milione di elementi. Se la passi "per riferimento", copia solo 1 indirizzo (8 byte).

Ripasso Visivo/Schematico:

Modalità Passaggio	**Cosa viene copiato nello Stack**	**Impatto e Costo Computazionale**
Per Valore (Copia)	Il *Right-value* (Il dato stesso)	Costo legato alla dimensione del dato (es. $\Theta(N^2)$ per matrici). Sicuro: gli originali non cambiano.
Per Riferimento	Il *Left-value* (L'indirizzo)	Costo costante $\Theta(1)$. Permette la modifica dell'originale.

09 Marzo 2026 — Puntatori e Semantica degli Assegnamenti

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale: Puntatore**[3] Può assumere un valore simbolico `nil` qualora non referenzi alcun indirizzo valido.

- **La soluzione:** Nel linguaggio AG, il dereferenziamento è denotato dal prefisso asterisco (`*`). Se X è un puntatore, X è l'indirizzo (il suo right-value come variabile), mentre $*X$ indica l'oggetto puntato in memoria. La semantica delle istruzioni di assegnamento genera costi asintotici radicalmente diversi:

  - **Assegnamento tra puntatori:** `X := Y`. Si copia il right-value di Y nel left-value di X. Entrambi i puntatori ora referenziano lo stesso indirizzo (aliasing). Il costo è rigorosamente $\Theta(1)$.

[3] **Puntatore:** Una variabile la cui estensione in memoria è fissa (determinata dall'architettura, es. 4 o 8 byte) e il cui contenuto è logicamente interpretato come un indirizzo di memoria che referenzia un'altra locazione.

– **Assegnamento tra oggetti puntati:** `*X := *Y`. Se $*X$ e $*Y$ sono oggetti complessi (es. matrici di ordine N), il calcolo richiede la copia fisica dei blocchi di memoria puntati. Il costo è proporzionale alla dimensione della struttura, es. $\Theta(N^2)$.

Guida di Studio Approfondita

Il Concetto Base: Un puntatore è semplicemente un foglietto di carta con scritto sopra un indirizzo (es. "Via Roma 2"). Il foglietto in sé occupa sempre la stessa quantità di spazio, indipendentemente dal fatto che a "Via Roma 2" ci sia un piccolo box auto o un gigantesco grattacielo (l'oggetto puntato).

Analisi Passo-Passo (L'asterisco magico):

1. `X := Y`: Prendo il foglietto Y, leggo l'indirizzo e lo ricopio sul foglietto X. Ora ho due foglietti che mi mandano alla stessa casa. È un'operazione istantanea.

2. `*X := *Y`: L'asterisco significa "vai all'indirizzo". Questa riga ordina al computer: "Vai alla casa indicata da Y, prendi tutti i mobili, vai alla casa indicata da X, e copia tutti i mobili lì dentro". È un'operazione estremamente pesante se la "casa" è piena di dati.

Il "Perché" è importante: Molti bug di prestazioni (o di logica) nei linguaggi orientati agli oggetti come Java derivano dall'ignorare questa differenza. In Java, quando scrivi `Oggetto A = B`, stai facendo un'operazione sui puntatori (riferimenti). Non stai clonando l'oggetto. Comprendere questa astrazione nativa è vitale per gestire la memoria dinamica e le strutture dati concatenate.

Ripasso Visivo/Schematico:

Istruzione	**Semantica Logica**	**Costo (es. su Matrici $N \times N$)**
`X := Y`	Copia dell'indirizzo. *Aliasing.*	$\Theta(1)$ (Costante)
`*X := *Y`	Copia fisica dei dati (Profonda).	$\Theta(N^2)$ (Quadratico)

09 Marzo 2026 Strutture Dati Elementari: Vettori, Matrici e Record

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale: Strutture Dati Elementari**[4]

- **La soluzione:** La mappa di memorizzazione è la formula analitica che il compilatore utilizza in tempo di esecuzione per tradurre indici logici in indirizzi fisici assoluti della RAM. Sia α l'indirizzo base della struttura e k la dimensione in byte del tipo base.

- **Vettore (1D):** Per accedere all'elemento logico i-esimo:

$$Indirizzo = \alpha + k \cdot (i - 1) \quad \text{(se gli indici partono da 1)}$$

$$Indirizzo = \alpha + k \cdot i \quad \text{(se gli indici partono da 0)}$$

[4] **Strutture Dati Elementari:** Modelli di organizzazione della memoria caratterizzati da allocazione statica contigua e tempi di accesso diretto costanti.

- **Matrice (2D):** Memorizzazione per righe (row-major) per una matrice $M \times N$:

$$Indirizzo(i, j) = \alpha + k \cdot (i \cdot N + j) \quad \text{(con indici 0-based)}$$

  L'indirizzamento è una computazione eseguita in tempo rigorosamente costante $\Theta(1)$, basata su un numero fisso di operazioni aritmetiche.

- **Record:** Struttura eterogenea a lunghezza fissa. Il calcolo dell'indirizzo per il campo i-esimo avviene *a tempo di compilazione* conoscendo le dimensioni k_j dei campi precedenti:

$$Indirizzo_Campo_i = \alpha + \sum_{j=1}^{i-1} k_j$$

Guida di Studio Approfondita

Il Concetto Base: La RAM è un lungo nastro continuo di cassetti numerati. Se crei un array (vettore), il computer riserva un blocco contiguo di cassetti. Se ogni elemento occupa k cassetti, per trovare l'elemento numero i, il computer non deve scorrere la memoria dall'inizio: fa un calcolo matematico diretto per "saltare" esattamente alla posizione giusta. Ecco perché l'accesso agli array è istantaneo, indipendentemente dalla grandezza dell'array.

Analisi Passo-Passo (Perché gli array partono da 0 in Java/C?):

1. Guarda la formula matematica se gli array partono da 1: $\alpha + k \cdot (i - 1)$. Il computer deve eseguire una sottrazione, una moltiplicazione e un'addizione.

2. Se l'array parte da 0, la formula è: $\alpha + k \cdot i$. Sparisce la sottrazione. Il calcolo è fisicamente più veloce nei circuiti del processore. L'indicizzazione zero-based è una pura ottimizzazione ingegneristica per l'efficienza.

Il "Perché" è importante: Questa teoria mostra la differenza tra Vettori (strutture omogenee e continue) e Record (strutture eterogenee usate nei database). Nelle matrici, sapere se il compilatore usa l'ordine "per righe" o "per colonne" è vitale per l'uso della cache moderna: accedere ai dati nell'ordine esatto in cui sono disposti in RAM rende i programmi immensamente più veloci (spatial locality).

Ripasso Visivo/Schematico:

Struttura	**Caratteristica Dati**	**Tempo di Accesso**
Vettore	Omogenei. Dimensione statica fissa.	$\Theta(1)$ (Eseguito a Runtime)
Matrice	Omogenei. Bidimensionale linearizzata.	$\Theta(1)$ (Eseguito a Runtime)
Record	Eterogenei (es. Stringhe + Interi).	$\Theta(1)$ (Calcolato in Compilazione)

09 Marzo 2026 — Introduzione alle Liste Concatenate

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale: Lista**[5] L'assenza di elementi genera la lista vuota, denotata dal simbolo Λ.

[5] **Lista:** Contenitore di informazioni uniforme, ma, a differenza del vettore, di dimensione dinamicamente variabile e non nota a priori. Formalmente definita come l'unione infinita di tutte le possibili sequenze finite di elementi di un certo tipo U ($\bigcup_{i=0}^{\infty} U^i$).

- **La soluzione:** La lista viene manipolata attraverso un set minimo di operazioni astratte primitive:
  - `isempty(L)`: Predicato booleano per verificare se $L = \Lambda$.
  - `elemento(L, k)`: Restituisce l'elemento in posizione k. Se k eccede la dimensione, restituisce indefinito ($\perp$).
  - `inserisci(L, k, u)`: Allunga la sequenza. Gli elementi da k a n shiftano nella posizione successiva.
  - `togli(L, k)`: Accorcia la sequenza. Gli elementi da $k+1$ a n shiftano nella posizione precedente.

  Nota: Durante la lezione, l'implementazione pratica in RAM (tramite puntatori e allocazione dinamica) viene posticipata alla lezione successiva, definendo per ora le operazioni solo a livello logico/algebrico.

Guida di Studio Approfondita

Il Concetto Base: L'array è come un vassoio di uova: compri il vassoio da 6, hai 6 spazi. Se te ne serve un settimo, devi buttare il vassoio e comprarne uno nuovo da 12. La Lista risolve questo problema strutturale: è come un treno a cui puoi agganciare e sganciare vagoni all'infinito durante il viaggio. Non ha un limite di dimensione imposto all'inizio.

Analisi Passo-Passo (Le Operazioni): L'inserimento e la cancellazione logica comportano uno slittamento relativo delle posizioni.

1. Immagina la classifica di una gara di sci. Se uno sciatore si inserisce al 3° posto (`inserisci`), chi era 3° diventa 4°, il 4° diventa 5°, ecc.

2. Se il 2° classificato viene squalificato (`togli`), il 3° scala al 2° posto, il 4° al 3°, ecc. La lunghezza totale diminuisce dinamicamente.

Il "Perché" è importante: La lista è il concetto matematico astratto che fonda tutte le strutture dati dinamiche. Rimuove la rigidità (Out of Bounds Exception) degli array. Tuttavia, come vedremo implementandola, rimuovere il limite di spazio ci farà perdere il privilegio del tempo di accesso istantaneo. L'informatica è sempre un compromesso (trade-off) tra flessibilità e velocità.

Ripasso Visivo/Schematico:

Proprietà	**Vettore (Array)**	**Lista Astratta**
Dimensione	Statica, fissata alla creazione.	Dinamica, teoricamente illimitata.
Posizione/Indice	Immutabile fisicamente in memoria.	Relativa, slitta dopo `inserisci` o `togli`.
Contenuto	Uniforme (stesso tipo base U).	Uniforme (stesso tipo base U).

Capitolo 8

Strutture Dati: Dalla Memoria Statica all'Allocazione Dinamica

10 Marzo 2026 Mappe di Memorizzazione e Ottimizzazione degli Indici

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale: Mappa di Memorizzazione**[1]
- **Memorizzazione per Righe (Row-Major)**[2]
- **La soluzione:** Il costo temporale per accedere a un elemento deve essere $\Theta(1)$. L'equazione di indirizzamento fisico per una matrice $M \times N$ con indici che partono da 1 ($1 \leq i \leq M, 1 \leq j \leq N$) richiede il salto delle prime $i-1$ righe e dei primi $j-1$ elementi sulla riga corrente:

$$Indirizzo(i,j) = \alpha + k \cdot \Big[(i-1) \cdot N + (j-1)\Big]$$

Dove α è l'indirizzo base e k è la dimensione in byte del tipo base. L'ottimizzazione architetturale impone l'uso di indici a partire da 0 (zero-based indexing), semplificando l'equazione di ricorrenza interna del calcolo a:

$$Indirizzo(i,j) = \alpha + k \cdot (i \cdot N + j)$$

Questa trasformazione riduce il numero di operazioni aritmetiche atomiche da 7 (3 moltiplicazioni, 2 somme, 2 sottrazioni) a 5 (3 moltiplicazioni, 2 somme).

Guida di Studio Approfondita

Il Concetto Base: La RAM del computer è un unico lungo "nastro" di cassetti. Una matrice (es. 10×10) è un oggetto 2D logico che deve essere "appiattito" per entrarci. Se cerchi l'elemento

[1] **Mappa di Memorizzazione:** Funzione matematica utilizzata dal compilatore per linearizzare una struttura dati multidimensionale in una memoria monodimensionale (RAM), traducendo indici logici in indirizzi fisici assoluti.

[2] **Memorizzazione per Righe (Row-Major):** Criterio di linearizzazione per matrici di ordine $M \times N$ in cui gli elementi della prima riga precedono quelli della seconda, e così via.

alla riga 5, colonna 3, il computer non scorre i cassetti uno a uno: calcola esattamente l'indirizzo matematico saltando le prime 4 righe intere. Questo garantisce che trovare il 1° o il 1000° elemento richieda sempre lo stesso identico tempo asintotico.

Analisi Passo-Passo (Perché gli array partono da 0): Nei linguaggi moderni (come C o Java) si parte da 0 per pura efficienza hardware.

1. Se parti da 1, la CPU deve calcolare $i - 1$ e $j - 1$ ogni singola volta che apri le parentesi quadre della matrice `M[i][j]`.

2. Se parti da 0, i "-1" scompaiono dalla formula. Risparmiare 2 sottrazioni per ogni accesso a una matrice, ripetuto miliardi di volte in un ciclo `for`, trasforma un calcolo teorico in un guadagno prestazionale netto (dal 33% di operazioni in meno).

Il "Perché" è importante: Capire come i dati vengono disposti in memoria (per righe o per colonne) è cruciale per la località spaziale delle cache moderne. Ma soprattutto, giustifica scelte sintattiche storiche dei linguaggi di programmazione, derivate da rigidi vincoli di ottimizzazione del ciclo Fetch-Decode-Execute.

Ripasso Visivo/Schematico:

Parametro	**Indici 1-based (es. Algol, Pascal)**	**Indici 0-based (es. C, Java)**
Formula Row-Major	$\alpha + k \cdot [(i-1)N + (j-1)]$	$\alpha + k \cdot (i \cdot N + j)$
Costo Calcolo RAM	7 operazioni aritmetiche	5 operazioni aritmetiche
Tempo d'Accesso	$\Theta(1)$	$\Theta(1)$ (più veloce in costante)

10 Marzo 2026 Implementazione delle Liste Concatenate tramite Record e Puntatori

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale: Lista Concatenata**[3]
- **Nodo**[4]
- **La soluzione:** L'accesso sequenziale viene forzato dalla struttura stessa. Mentre un vettore mappa l'indice i in un indirizzo a tempo $\Theta(1)$, la ricerca di un elemento (es. predicato booleano `Appartiene`) o l'accesso posizionale richiedono la dereferenziazione successiva dei puntatori. Il costo dell'attraversamento (Traversal) obbedisce all'equazione:

$$T(n) = \sum_{k=1}^{i} \Theta(1) = \Theta(i)$$

  Inserire un elemento in testa richiede invece un tempo strettamente costante, $\Theta(1)$, tramite riassegnamento di riferimenti: la variabile che punta al primo nodo viene aggiornata per puntare al nodo neocreato, il cui campo `next` punterà al vecchio primo nodo.

[3]**Lista Concatenata:** Implementazione dinamica della struttura logica Lista, in cui lo spazio non è allocato in modo contiguo. Formata da oggetti distribuiti arbitrariamente nello spazio di indirizzamento.

[4]**Nodo:** Entità base della lista implementata come Record eterogeneo a due campi: `el` (campo informativo di tipo U) e `succ/next` (campo puntatore contenente l'indirizzo del nodo successivo).

Guida di Studio Approfondita

Il Concetto Base: A differenza del vettore (che è un blocco di case a schiera), la lista concatenata è una caccia al tesoro. Hai l'indirizzo della prima casa. Dentro la prima casa (il *nodo*) trovi il dato che ti serve (es. un intero) e un foglietto con l'indirizzo della seconda casa (*puntatore*). Continui a seguire i foglietti finché non trovi un foglietto vuoto (`nil` o `null`), che indica la fine del percorso.

Analisi Passo-Passo (Operazioni a costo variabile):

1. **Ricerca:** Per sapere se un elemento c'è, devi bussare alla prima casa. Se non è lì, leggi l'indirizzo della seconda, ci vai, bussi. Nel caso peggiore, l'elemento non c'è, e hai visitato tutte le n case. Costo: lineare, $\Theta(n)$.

2. **Inserimento in testa:** Vuoi aggiungere una casa all'inizio. Costruisci la nuova casa. Metti nel foglietto della nuova casa l'indirizzo della "vecchia" prima casa. Aggiorni il punto di partenza per indicare la tua nuova casa. Non hai dovuto spostare nient'altro. Costo: istantaneo, $\Theta(1)$.

Il "Perché" è importante: Il trade-off fondamentale dell'informatica: Vettori offrono un accesso istantaneo ($\Theta(1)$) ma non possono crescere o rimpicciolirsi senza costi enormi (copia dell'intero blocco). Le liste offrono crescita/riduzione a costo nullo, ma sacrificano l'accesso istantaneo ($\Theta(n)$). Capire questo significa poter scegliere la struttura dati adatta allo specifico problema software che si sta modellando.

Ripasso Visivo/Schematico:

Operazione	Vettore (Memoria Contigua)	Lista Concatenata (Nodi/Puntatori)
Accesso per Indice	$\Theta(1)$ (Matematico)	$\Theta(n)$ (Attraversamento lineare)
Inserimento in Testa	$\Theta(n)$ (Shift totale a destra)	$\Theta(1)$ (Riassegnamento 2 puntatori)
Dimensione Memoria	Rigida (Limita l'Hardware)	Dinamica (Fino a esaurimento Heap)

10 Marzo 2026 Gestione Rudimentale della Memoria: La Free List

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale: Memory Manager**[5]
- **Free List**[6]
- **La soluzione:** Assumendo blocchi di dimensione uniforme (es. partizioni costanti, come banconote di taglio fisso), l'allocazione (`Alloca`) e la deallocazione (`Libera`) si riducono a elementari operazioni sulla testa della lista, eseguibili in tempo costante. L'equazione logica per `Alloca`:

$$Blocco := Testa(FreeList); \quad FreeList := FreeList.Next$$

[5]**Memory Manager:** Sottosistema che deve allocare spazio in modo dinamico a fronte di richieste a tempo di esecuzione, laddove i linguaggi privi di Garbage Collector (es. C) lasciano tale responsabilità allo sviluppatore.

[6]**Free List:** Struttura dati (lista concatenata) utilizzata a basso livello per mantenere traccia dei blocchi di memoria fisicamente contigui, liberi e allocabili. Ciascun nodo della Free List rappresenta il punto di accesso a un blocco (chunk) di memoria fissa predimensionata.

L'allocazione avviene scorporando il primo blocco libero (aggiornando il puntatore della `FreeList` al suo `succ`) e fornendo al programma chiamante (Client) l'indirizzo base del blocco estratto (costo $\Theta(1)$).

Guida di Studio Approfondita

Il Concetto Base: Pensa a un gestore della memoria come a uno sportello Bancomat che può erogare solo banconote da 50 euro (blocchi di memoria di dimensione fissa). Il Bancomat ha una cassa (la *Free List*) dove le banconote sono impilate. Quando un programma richiede memoria ("voglio un prestito"), il Bancomat prende la primissima banconota in cima alla pila, la cancella dalla sua lista interna e te la dà. Quando non ti serve più, la restituisci al Bancomat, che la rimette in cima alla pila.

Analisi Passo-Passo (La procedura Alloca):

1. **Controllo Sicurezza:** Se la `FreeList` è `nil` (null), il Bancomat è vuoto. Il sistema va in "Out of Memory" (errore in esecuzione, $\perp$).

2. **Estrazione:** Se ci sono blocchi, il gestore stacca il primo blocco dalla `FreeList` (aggiornando il puntatore d'inizio della lista libera).

3. **Assegnazione:** L'indirizzo di quel blocco viene collegato alla "lista dei blocchi in uso" dell'applicazione che ne ha fatto richiesta. L'operazione è istantanea perché non devo cercare un blocco specifico: sono tutti uguali (taglio fisso da 50 euro), quindi il primo va sempre bene.

Il "Perché" è importante: Questo modello base svela cosa succede "sotto il cofano" del comando `new` in Java o `malloc` in C. Mentre Java fa pulizia da solo con il *Garbage Collector* (che ha un costo prestazionale a tempo di esecuzione), qui tocchiamo il concetto puro: la memoria è un prestito che si consuma. Se prelevi sempre e non rimetti mai nulla nella Free List (Memory Leak), l'applicazione prima o poi terminerà inevitabilmente con un blocco del sistema.

Ripasso Visivo/Schematico:

Evento	**Stato della Free List**	**Azione Logica**
Programma fa `Alloca()`	Perde il primo nodo (Pop). La lista si accorcia.	Ritorna l'indirizzo al programma. Costo $\Theta(1)$.
Programma restituisce blocco	Guadagna un nodo in testa (Push). La lista si allunga.	Riaggiunge lo spazio alla cassa. Costo $\Theta(1)$.
`FreeList == nil`	Il sistema non ha più blocchi contigui disponibili.	Errore "Out of Memory". Computazione bloccata.

Capitolo 9

Strutture Dati Dinamiche: Liste e Gestione della Memoria

12 Marzo 2026 — Definizione Astratta e Operazioni sulle Liste

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale: Lista**[1] La lista priva di elementi è denominata *lista vuota* e indicata con il simbolo Λ.
- L'interfaccia astratta della struttura dati si fonda su operazioni primitive:
  - `isempty(L)`: Ritorna *vero* se $L = \Lambda$, *falso* altrimenti.
  - `elemento(L, k)`: Restituisce l'elemento in posizione k. Se k eccede i confini, restituisce un errore/indefinito ($\perp$).
  - `inserisci(L, k, u)`: Aggiunge l'elemento u in posizione k, slittando gli elementi preesistenti da k a n verso posizioni successive.
  - `togli(L, k)`: Rimuove l'elemento in posizione k, ricompattando la lista (slittamento all'indietro degli elementi da $k+1$ a n).
- **La soluzione:** La manipolazione complessa avviene tramite combinazione delle operazioni primitive o definizioni ricorsive. Un esempio fondamentale è il calcolo della lunghezza di una lista, che ammette la seguente specifica formale ricorsiva:

$$LUNGHEZZA(L) = \begin{cases} 0 & \text{se } L = \Lambda \\ 1 + LUNGHEZZA(togli_in_testa(L)) & \text{altrimenti} \end{cases}$$

  Allo stesso modo, la funzione per contare le occorrenze di un elemento A in una lista L valuta se il primo elemento coincide con A (sommando 1) o meno (sommando 0), sommandovi il numero di occorrenze nel resto della lista.

[1] **Lista:** Contenitore di informazione uniforme atto a rappresentare sequenze di dati dello stesso tipo base U, privo di un limite dimensionale prefissato. Algebricamente definita come l'unione infinita di tutte le potenze dell'insieme U ($\bigcup_{i=0}^{\infty} U^i$).

Guida di Studio Approfondita

Il Concetto Base: A differenza del vettore (che nasce e muore con una dimensione fissa), la lista è elastica. Immagina la classifica di una gara di sci. Se un nuovo sciatore si qualifica al 7° posto (`inserisci`), non si sovrascrive chi era 7°, ma tutti gli sciatori dalla 7° posizione in poi "scalano" di una posizione verso il basso. Se uno sciatore viene squalificato (`togli`), chi gli stava dietro avanza di un posto.

Analisi Passo-Passo (Lunghezza Ricorsiva): Come calcoliamo la lunghezza senza un contatore prefissato?

1. **Caso Base:** Se la lista è vuota, la lunghezza è banalmente 0.
2. **Passo Ricorsivo:** Se la lista non è vuota, prendiamo il primo elemento (la *testa*). Lo contiamo come "1". A questo 1 sommiamo la lunghezza di tutto il resto della lista (la *coda*).
3. Esempio visivo: Lista $[A, B, C]$. Lunghezza = 1 (per A) + Lunghezza di $[B, C]$. Che a sua volta è 1 (per B) + Lunghezza di $[C]$. Che è 1 (per C) + Lunghezza di [] (che è 0). Totale: $1 + 1 + 1 + 0 = 3$.

Il "Perché" è importante: Definire le operazioni in astratto, senza sapere come il computer le memorizzerà fisicamente, permette di progettare la logica (la semantica) in modo puro. L'assenza della primitiva "accesso in scrittura" (modifica diretta) ci insegna che possiamo derivarla: per modificare una posizione k, basta inserire il nuovo dato in k e cancellare il vecchio dato scivolato in $k + 1$.

Ripasso Visivo/Schematico:

Operazione	Effetto Logico	Comportamento su Posizioni
`inserisci(L, k, u)`	Allunga la sequenza	Slittamento in avanti ($i \to i + 1$)
`togli(L, k)`	Accorcia la sequenza	Slittamento all'indietro ($i + 1 \to i$)
`elemento(L, k)`	Sola lettura	Nessuna modifica strutturale

12 Marzo 2026 Implementazione delle Liste: Vettori Paralleli e Puntatori

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale:** Passare dalla specifica algebrica della lista alla sua rappresentazione all'interno della memoria RAM, la quale richiede la conservazione non solo dei dati, ma anche della struttura sequenziale (l'ordine relativo degli elementi).
- **Vettori Paralleli**[2] Il valore 0 in `succ` convenzionalmente codifica la fine della lista (Λ).
- **Implementazione a Puntatori (Lista Concatenata)**[3] L'indirizzo speciale `nil` denota l'assenza di successore.

[2] **Vettori Paralleli:** Implementazione statica che simula la dinamicità. Utilizza due vettori della medesima dimensione N: il vettore `elem` per i dati e il vettore `succ` per gli indici. Per un elemento in posizione fisica i, `succ[i]` contiene l'indice fisico del prossimo elemento logico della lista.

[3] **Implementazione a Puntatori (Lista Concatenata):** Rappresentazione puramente dinamica in cui la lista è un puntatore a un *Nodo*. Un nodo è un record eterogeneo composto da un campo informativo (`item`) e un campo puntatore (`next`) contenente l'indirizzo assoluto in RAM del nodo successivo.

- **La soluzione:** La ricerca di un elemento (`trova`) o l'accesso a una determinata posizione logica k impongono un attraversamento sequenziale forzato. L'equazione asintotica per la scansione lineare è:

$$T(n) = O(n)$$

Di contro, le operazioni eseguite alle estremità note, come l'inserimento in testa su una lista concatenata, necessitano della pura riassegnazione di riferimenti:

1. Allocazione nuovo nodo X.
2. `X.next = L` (il nuovo nodo punta al vecchio primo nodo).
3. `L = X` (la testa della lista viene aggiornata al nuovo nodo).

L'equazione di costo per l'inserimento in testa diventa rigorosamente $\Theta(1)$.

Guida di Studio Approfondita

Il Concetto Base: Per mantenere l'ordine in una lista senza metterli fisicamente vicini in memoria, ci serve un trucco. Metodo 1 (Vettori Paralleli): Usiamo due array. Nel primo ci sono i dati sparsi a caso. Nel secondo ci sono gli "indici di salto". Leggo in posizione 4 la lettera 'B', guardo il secondo array in posizione 4 e trovo '5'. Salto alla posizione 5 per trovare il prossimo dato. Metodo 2 (Puntatori): Ogni dato vive in una casa isolata. Ma dentro la casa, oltre al dato, c'è un foglietto di carta (*puntatore*) con scritto l'indirizzo esatto della prossima casa. Seguo i foglietti finché non ne trovo uno bianco (`nil`).

Analisi Passo-Passo (Costo della Ricerca vs Inserimento in Testa):

1. **Ricerca $O(n)$:** Nelle liste concatenate, non esiste l'accesso matematico diretto come negli array. Se voglio l'elemento 1000, devo bussare alla casa 1, leggere il foglietto per la 2, andare alla 2, leggere per la 3, e così via per 1000 volte. Se l'elemento non c'è (caso peggiore), attraverso tutti gli n elementi.

2. **Inserimento in Testa $\Theta(1)$:** Vuoi aggiungere un vagone in testa al treno? Costruisci il vagone. Attacchi il suo gancio al vecchio primo vagone. Dici al capostazione che il nuovo inizio del treno è questo nuovo vagone. Hai fatto 3 mosse indipendentemente dalla lunghezza totale del treno. Costo costante.

Il "Perché" è importante: I vettori paralleli permettono di avere liste multiple intrecciate nello stesso spazio contiguo (limitato però a N elementi globali). L'implementazione a puntatori rimuove ogni limite fisico pre-impostato, ma sacrifica le prestazioni in lettura per guadagnare eccellenti prestazioni ($\Theta(1)$) in modifica alle estremità.

Ripasso Visivo/Schematico:

Caratteristica	**Vettori Paralleli**	**Record e Puntatori**
Allocazione	Statica (dimensione Max N bloccata)	Dinamica (cresce a runtime)
Formato Link	Indice numerico (es. 5)	Indirizzo di memoria RAM (es. `0xFA12`)
Terminatore	Valore 0 (convenzione)	`nil` / `null`

12 Marzo 2026 — Gestore della Memoria: La Free List

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale: Memory Manager (Allocatore)**[4]
- **Free List**[5] Nell'assunzione del modello semplificato trattato, ogni blocco ha una dimensione fissa e predeterminata.
- **La soluzione:** La logica di erogazione (`Alloca`) di un blocco di memoria in tale modello non prevede ricerca, ma una pura estrazione in testa. L'algoritmo di astrazione procede come segue:

  1. Valutazione disponibilità: Se `FreeList = nil`, generazione di eccezione *Out of Memory* (la memoria è saturata).
  2. Estrazione: Se vi è disponibilità, si salva il riferimento all'elemento in testa: $X = FreeList$.
  3. Aggiornamento: La Free List slitta al blocco successivo: $FreeList = FreeList.succ$.
  4. Inserimento nel chiamante: Il blocco referenziato da X viene inserito in testa all'interno della lista dei blocchi in uso dall'applicazione.

  La complessità dell'operazione `Alloca` è limitata da un tempo rigorosamente costante:

$$T_{Alloca}(n) = \Theta(1)$$

  Nota: L'implementazione della logica speculare di deallocazione/versamento (Libera) è frammentaria nella lezione corrente ed è stata esplicitamente rimandata alla lezione successiva dal docente.

Guida di Studio Approfondita

Il Concetto Base: I linguaggi senza *Garbage Collector* devono gestire lo spazio a mano. Pensa a un "Bancomat" speciale che distribuisce solo banconote di taglio fisso (es. 50€). Il bancomat ha una cassa interna, la *Free List*, in cui tutte le banconote disponibili sono impilate. Quando un programma chiede memoria ("voglio un prestito"), il bancomat non perde tempo a cercare la banconota più bella: prende esattamente la prima in cima alla pila, la cancella dal suo elenco e la consegna al programma.

Analisi Passo-Passo (La procedura Alloca):

1. **Sicurezza prima di tutto:** Se il bancomat è vuoto (`FreeList == nil`), l'operazione abortisce. Il computer si ferma e restituisce il classico errore di sistema (codice di errore o crash).
2. **Prelevo (Pop):** Prendo il primo blocco libero e aggiorno la Free List facendola iniziare dal secondo blocco.
3. **Assegnazione (Push):** Prendo il blocco appena scollegato dal bancomat e lo "aggancio" come primo blocco nella lista personale dell'applicazione che lo ha richiesto. Tutto avviene agendo unicamente sulle *teste* delle liste, istantaneamente.

[4]**Memory Manager (Allocatore):** Sottosistema incaricato di gestire lo spazio libero in RAM, fornendo blocchi contigui alle applicazioni chiamanti in fase di esecuzione.

[5]**Free List:** Struttura dati (tipicamente una lista concatenata) mantenuta a basso livello dal gestore della memoria, in cui i nodi non contengono dati applicativi, ma rappresentano blocchi (chunk) di memoria fisicamente liberi.

Il "Perché" è importante: Questa astrazione ci fa comprendere il costo celato dietro l'istruzione `new` degli oggetti. Mentre in linguaggi come Java l'ambiente gestisce automaticamente la restituzione dei blocchi tramite il Garbage Collector (evitando di lasciare blocchi irraggiungibili, detti "orfani"), nei sistemi a basso livello estrarre blocchi senza mai rimetterli nella *Free List* porta inesorabilmente alla saturazione del sistema (Memory Leak).

Ripasso Visivo/Schematico:

Stato Free List	**Azione del Memory Manager**	**Costo**
`FL == nil`	Errore: Memoria Esaurita. Abortisce.	$\Theta(1)$
`FL != nil`	Stacca nodo in testa (Pop) e passa riferimento.	$\Theta(1)$

Capitolo 10

Strutture Dati Dinamiche e ad Accesso Ristretto

16 Marzo 2026 — Il Problema di Giuseppe Flavio e le Liste Circolari

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale: Problema di Giuseppe Flavio**[1]
- **Lista Circolare**[2]
- **La soluzione:** Sono proposti due schemi algoritmici basati su strutture dati diverse.
- **Implementazione con Lista Circolare:** Si costruisce una lista circolare di N nodi con costo $\Theta(N)$. L'algoritmo avanza di $M-1$ passi, elimina il nodo successivo e riallaccia i puntatori. L'eliminazione avviene $N-1$ volte. L'equazione di costo totale per la simulazione è:
$$T(N, M) = \Theta(N) + \Theta(N \cdot M) = \Theta(N \cdot M)$$
Se M è una costante limitata (es. $M \ll N$), la complessità degrada a un tempo asintotico lineare $\Theta(N)$.
- **Implementazione con Vettore (Soluzione Naive):** Utilizzando un array di dimensione statica, le eliminazioni si effettuano segnando logicamente un elemento come "morto" (es. valore simbolico speciale). Poiché lo spazio fisico non si riduce, il conteggio modulo M costringe ad attraversare iterativamente anche gli elementi già eliminati. Quando la struttura è semi-vuota, lo scorrimento degli elementi nulli porta la complessità a un comportamento quadratico:
$$T(N) = \Theta(N^2)$$
Questa divergenza asintotica dimostra la superiorità della lista circolare per questo specifico problema.

[1] **Problema di Giuseppe Flavio:** Dato un insieme di N entità disposte in cerchio e un intero di conteggio M, si procede all'eliminazione ciclica dell'entità in posizione M (contando a partire dall'ultimo eliminato). Determinare l'algoritmo ottimo per individuare l'ultimo superstite.

[2] **Lista Circolare:** Estensione logica della lista concatenata semplice in cui il puntatore dell'ultimo elemento (la coda) non assume il valore speciale `nil`, ma contiene il riferimento al primo elemento (la testa), creando una topologia ad anello.

Guida di Studio Approfondita

Il Concetto Base: Immaginiamo N persone sedute in cerchio che fanno una "conta" di M passi. Chi finisce la conta viene eliminato. Si continua a contare a partire dal successivo finché non ne rimane uno solo. Se usiamo una lista circolare (come tenersi per mano), quando una persona viene eliminata, le due ai suoi fianchi si danno la mano: il cerchio si stringe fisicamente. Se usiamo un array, le sedie rimangono fisse: chi è eliminato lascia la sedia vuota. Chi conta deve "saltare" le sedie vuote, perdendo un'enormità di tempo man mano che le sedie si svuotano.

Analisi Passo-Passo (Lista Circolare):

1. **Costruzione:** Creo i nodi da 1 a N. Il campo puntatore del nodo N viene forzato a puntare al nodo 1.
2. **Avanzamento:** Per eliminare il nodo M-esimo, mi muovo con un ciclo `for` fermandomi al nodo $M-1$ (il nodo precedente).
3. **Bypass (Eliminazione):** Prendo il puntatore del nodo $M-1$ e lo collego direttamente al nodo $M+1$. Il nodo M è isolato (diventa "orfano") e il Garbage Collector lo rimuoverà dalla memoria.

Il "Perché" è importante: Questo problema non è un banale gioco matematico. Dimostra una regola aurea dell'informatica: *scegliere la struttura dati sbagliata trasforma un algoritmo concettualmente corretto in un disastro computazionale.* L'approccio con vettore crolla in $\Theta(N^2)$, mentre l'approccio con lista circolare vince in $\Theta(N \cdot M)$.

Ripasso Visivo/Schematico:

Struttura Dati	**Gestione dell'eliminazione**	**Complessità Computazionale**
Vettore (Array)	Cancellazione logica. Gli elementi "vuoti" devono essere attraversati.	$\Theta(N^2)$ (Inaccettabile per input elevati)
Lista Circolare	Cancellazione fisica. I puntatori si riallacciano, aggirando il nodo.	$\Theta(N \cdot M)$ (Spesso approssimabile a lineare)

16 Marzo 2026 — La Pila (Stack) e logica LIFO

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale: Pila (Stack)**[3]
- **Operazioni Primitive (Specifica Algebrica):**
  - `isempty(S)`: Predicato booleano; vero se la pila è vuota (Λ).
  - `top(S)`: Restituisce l'elemento in cima alla pila senza alterarne la struttura. Indefinito ($\perp$) se $S = \Lambda$.
  - `pop(S)`: Modifica la pila rimuovendo l'elemento in cima. Indefinito se $S = \Lambda$. Spesso implementata come funzione ibrida che estrae e restituisce il valore.

[3] **Pila (Stack):** Specifica di una struttura dati dinamica ad accesso ristretto che adotta una politica di servizio LIFO (Last In, First Out). Modella l'astrazione di una sequenza in cui inserimenti e rimozioni possono avvenire esclusivamente a una singola estremità, denominata *cima* (top).

- `push(S, a)`: Modifica la pila inserendo l'elemento a come nuova cima.

- **La soluzione (Implementazioni in RAM):**

  1. **Implementazione con Lista Concatenata:** La cima della pila coincide convenzionalmente con il primo elemento (testa) della lista. La `push` equivale a inserire in testa; la `pop` equivale a togliere in testa. Il costo per singola operazione è rigorosamente $\Theta(1)$.
  2. **Implementazione con Vettore (Array):** Richiede un array di dimensione statica N e un registro indice `top` che punta alla prima cella libera. Condizione di pila vuota: `top == 0`. Condizione di pila piena (Overflow): `top == N`. L'inserimento oltre N non è ammesso. Se vi è disponibilità, tutte le primitive vengono eseguite manipolando gli indici in tempo costante $\Theta(1)$.

Guida di Studio Approfondita

Il Concetto Base: Pensa al distributore di vassoi della mensa (quello a molla cilindrica). L'ultimo vassoio che l'operatore impila (`push`) è esattamente il primo vassoio che il cliente successivo andrà a prendere (`pop`). Non puoi estrarre il vassoio in fondo senza prima rimuovere tutti quelli sopra di esso. Questo è il principio LIFO.

Analisi Passo-Passo (Implementazione tramite Array): Visto che l'array è una scatola rigida, per simulare una Pila usiamo una variabile "segnalibro" chiamata `top`.

1. Se `top = 0`, lo stack è vuoto.
2. Se eseguo `push(valore)`, l'array scrive il `valore` nella cella indicata da `top` e poi il segnalibro sale (`top = top + 1`).
3. Se eseguo `pop()`, il segnalibro scende (`top = top - 1`). L'elemento "cancellato" resta fisicamente scritto nella memoria RAM, ma è logicamente sparito: la prossima `push` lo sovrascriverà senza pietà.

Il "Perché" è importante: Lo Stack è il cuore pulsante dei computer moderni. Serve a gestire la ricorsione, l'annullamento delle azioni (l'operazione "Undo" nei word processor) e a invertire l'ordine dei dati (se inserisco A-B-C, estraggo C-B-A). L'implementazione a vettore è velocissima, ma richiede di conoscere in anticipo l'altezza massima (Out of Memory); la lista evita il blocco di memoria piena ma consuma overhead di istanziazione di nuovi oggetti.

Ripasso Visivo/Schematico:

Operazione	Costo (Lista)	Costo (Array)	Vincoli
`push`	$\Theta(1)$	$\Theta(1)$	Rischio Overflow se si usa un Array
`pop`	$\Theta(1)$	$\Theta(1)$	Errore (Underflow) se lo stack è vuoto
`top`	$\Theta(1)$	$\Theta(1)$	Sola lettura

16 Marzo 2026 — La Coda (Queue) e logica FIFO

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale: Coda (Queue)**[4]
- *Nota sulle registrazioni: Il docente accenna al concetto avanzato di "Coda con priorità" (facendo l'esempio del Triage ospedaliero in pronto soccorso con codici rossi, gialli, verdi e bianchi), tuttavia l'argomento è solo introduttivo e frammentario. L'implementazione tecnica verrà affrontata nelle lezioni successive. La trattazione si concentra qui sulle code standard.*
- **Operazioni Primitive:**
  - `isempty(Q)`: Vero se $Q = \Lambda$.
  - `front(Q)`: Restituisce l'elemento in testa senza rimuoverlo.
  - `dequeue(Q)`: Elimina l'elemento più "anziano" in testa alla coda.
  - `enqueue(Q, a)`: Accoda l'elemento a al termine della struttura.
- **La soluzione (Implementazione mediante Vettore Circolare):** L'implementazione su array richiede due indici: `first` (punta alla testa) e `last` (punta alla prima cella libera in coda). Per evitare lo sfondamento asimmetrico del vettore, lo scorrimento dell'indice in avanzamento avviene tramite aritmetica modulare di base N:

  $$indice_{nuovo} = (indice_{vecchio} + 1) \pmod N$$

  Tale approccio causa un'anomalia di ambiguità: se `first == last`, la coda è considerata vuota. Tuttavia, se il vettore si riempisse totalmente, `last` "doppierebbe" e si sovrapporrebbe a `first`, generando l'impossibilità di distinguere la coda satura da quella vuota. Si introduce un limite ingegneristico: un vettore di dimensione N può ospitare al massimo $N - 1$ elementi. Condizione di coda piena:

  $$(last + 1) \pmod N == first$$

  Il costo computazionale per ogni primitiva si assesta rigorosamente a $\Theta(1)$.

Guida di Studio Approfondita

Il Concetto Base: A differenza della pila, la coda è lo sportello delle poste: se entri per primo (First In), esci e vieni servito per primo (First Out). Chi arriva dopo si mette "in coda" dietro a tutti. Nessun salto di fila è permesso.

Analisi Passo-Passo (Il trucco del Vettore Circolare): Se usassimo un array normale (da sinistra a destra), aggiungendo persone a destra ed eliminando persone a sinistra, dopo un po' l'array scorrerebbe come un "bruco" fino a sbattere contro il fondo della memoria (Out of Bounds), pur avendo lasciato spazio vuoto all'inizio.

1. **Aritmetica Modulare (Modulo):** Quando l'indice `last` raggiunge l'ultima cella fisica (es. indice 9 su 10), l'operazione `(9+1) modulo 10` genera il risultato `0`. L'indice "rimbalza" istantaneamente all'inizio dell'array riutilizzando gli spazi liberati dalle precedenti estrazioni. L'array diventa un anello in cui le due "teste" si inseguono all'infinito.

2. **Il Sacrificio di uno Slot:** Affinché l'anello funzioni, un "posto sedia" deve rimanere perennemente vuoto. Serve come cuscinetto "anti-scontro" per non confondere una coda piena zeppa di persone con una coda totalmente svuotata.

[4] **Coda (Queue):** Specifica di una struttura dati dinamica ad accesso ristretto che adotta una politica di servizio FIFO (First In, First Out). Gli inserimenti avvengono esclusivamente a una estremità (coda o retro), mentre le estrazioni avvengono dall'estremità opposta (testa o fronte).

Il "Perché" è importante: La coda è essenziale nei sistemi operativi per gestire le stampanti (lo spooler di stampa), per la gestione dei task sui processori e nelle reti di telecomunicazione per instradare i pacchetti. Anche qui, l'implementazione array (vettore circolare) garantisce alte prestazioni ma vincola a una portata massima (es. coda da max 100 utenti).

Ripasso Visivo/Schematico:

Stato Coda	**Valutazione Matematica su Vettore Circolare**	**Azione Possibile**
Vuota	`first == last`	Sola Inserzione (Enqueue)
Piena	`(last + 1) mod N == first`	Sola Estrazione (Dequeue)
Normale	`first != last` (non è piena)	Entrambe consentite

Capitolo 11

Strutture Dati Dinamiche e Grafi: Implementazioni e Teoria degli Alberi

17 Marzo 2026 Implementazione Avanzata: Inserimento e Cancellazione posizionale nelle Liste

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale: Manipolazione Posizionale**[1]
- L'assunzione alla base dell'implementazione tramite record e puntatori è che non esista un accesso diretto posizionale. Pertanto, l'algoritmo deve attraversare sequenzialmente la struttura per raggiungere il punto di manipolazione.
- **La soluzione:** Per l'inserimento o la cancellazione in posizione i, la procedura deve arrestarsi sul nodo precedente, ovvero in posizione $i-1$, mantenendo un riferimento (es. variabile `pred`). Nel caso dell'inserimento, il campo puntatore del nuovo nodo punterà al successore di `pred` (`pred.next`), e il campo `next` di `pred` verrà aggiornato per puntare al nuovo nodo. Nel caso della cancellazione, il nodo in posizione i viene bypassato collegando `pred.next` direttamente a `pred.next.next`. I casi in cui $i = 1$ (testa della lista) rappresentano singolarità algoritmiche che richiedono la modifica esplicita del puntatore d'accesso principale (`first`). La complessità asintotica per la manipolazione al passo i obbedisce alla relazione:
$$T(i) = O(i)$$
Questo evidenzia il degrado prestazionale lineare all'allontanarsi dall'estremità della lista.

Guida di Studio Approfondita

Il Concetto Base: Inserire un vagone in mezzo a un treno in movimento (una lista concatenata) richiede cautela. Non puoi semplicemente "far apparire" il vagone. Devi prima percorrere i vagoni uno a uno partendo dalla locomotiva finché non trovi il punto esatto di giuntura (il nodo $i-1$).

[1] **Manipolazione Posizionale:** L'alterazione della struttura di una lista concatenata in una posizione logica i arbitraria, che richiede la gestione dinamica dei riferimenti di memoria tra il nodo in posizione $i-1$ e il nodo in posizione $i+1$.

Lì sganci i due vagoni vecchi, attacchi la parte posteriore del treno al tuo nuovo vagone, e poi attacchi il tuo nuovo vagone alla parte anteriore.

Analisi Passo-Passo (Inserimento in posizione generica i):

1. **Avanzamento:** Un ciclo `while` scorre la lista finché non raggiunge la posizione $i - 1$. Variabile di appoggio: `pred`.
2. **Ancoraggio Destro:** Il nuovo nodo viene istanziato. La sua "freccia" (`next`) viene impostata per indicare il nodo che attualmente si trova in posizione i (accessibile tramite `pred.next`).
3. **Ancoraggio Sinistro:** La freccia del nodo `pred` viene staccata e ridirezionata verso il nuovo nodo. L'elemento è ora agganciato saldamente.
4. **Gestione Testa:** Se $i = 1$, il ciclo di scorrimento non parte. Il nuovo nodo viene semplicemente attaccato all'attuale testa, e la variabile globale che indica l'inizio della lista si sposta sul nuovo nodo.

Il "Perché" è importante: Dal punto di vista software ingegneristico, omettere il controllo sui limiti (es. tentare di inserire in posizione 100 in una lista di 5 elementi) genera errori critici a runtime (Null Pointer Exception). Inoltre, questo evidenzia chiaramente la debolezza della lista: l'accesso ai dati in mezzo alla struttura costa $O(i)$, al contrario degli array che impiegano $\Theta(1)$.

Ripasso Visivo/Schematico:

Azione	**Costo Asintotico**	**Gestione Puntatori**
Testa ($i = 1$)	$O(1)$	Aggiornamento puntatore `first`
Generico ($i > 1$)	$O(i)$	Scorrimento lineare + aggancio `pred`
Cancellazione ($i = 1$)	$O(1)$	`first = first.next`

17 Marzo 2026 — Definizione e Tassonomia degli Alberi

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale: Albero**[2]
- **Albero con Radice**[3]
- **La soluzione:** La categorizzazione continua imponendo ulteriori vincoli:
  - **Albero k-ario:** Nessun nodo può possedere un grado di uscita (numero di figli) superiore a k, ed esiste almeno un nodo con grado esattamente pari a k. Il massimo numero di nodi a un livello i è regolato dall'equazione $N_{max}(i) = k^i$.
  - **Albero con radice ordinato:** Ogni classe di equivalenza (gruppo massimale di fratelli che condividono lo stesso padre) è dotata di un ordine totale intrinseco (es. primo fratello, secondo, ecc.).

[2] **Albero:** Modello matematico astratto definibile in tre modi funzionalmente equivalenti: 1) Un grafo non orientato, connesso e aciclico; 2) Un grafo non orientato in cui per ogni coppia di vertici esiste un unico cammino semplice che li unisce; 3) Un grafo non orientato e connesso avente n vertici e $n - 1$ lati.

[3] **Albero con Radice:** Una struttura ottenuta da un albero libero designando un vertice specifico come radice. Tale scelta induce una struttura gerarchica e direzionale dall'alto verso il basso, permettendo la definizione di livelli (lunghezza del cammino dalla radice) e relazioni di parentela (padre, figlio, fratello, antenato, discendente).

– **Albero binario:** Albero di grado massimo 2 in cui i figli sono spazialmente e logicamente distinti in "sinistro" e "destro". *Nota bene: la distinzione spaziale rende due alberi binari disuguali anche se topologicamente isomorfi, se un figlio unico è posizionato a sinistra in un caso e a destra nell'altro.*

Guida di Studio Approfondita

Il Concetto Base: Immagina un insieme di palline (nodi) collegate da fili (lati). Se sollevi l'intera struttura prendendo una singola pallina tra le dita, nessuna pallina cadrà a terra (perché il grafo è "connesso") e tutti i fili si tenderanno verso il basso senza formare anelli chiusi ("aciclico"). La pallina che tieni tra le dita è la *radice*. Da lì in giù, si forma una gerarchia rigida simile a un albero genealogico: chi sta sopra è "padre", chi sta sotto è "figlio", e chi non ha fili pendenti sotto di sé è una "foglia" (nodo esterno).

Analisi Passo-Passo (Equivalenza delle tre definizioni): Perché usare tre definizioni matematiche per la stessa cosa? Per comodità nelle dimostrazioni algoritmiche:

1. **Grafo aciclico connesso:** Se ci fosse un ciclo, sollevando la radice ci sarebbero due strade per arrivare alla stessa pallina sottostante, violando la gerarchia stretta.

2. **Cammino unico per ogni coppia:** Essendo connesso, esiste una strada. Essendo aciclico, se ne esistessero due, formerebbero un ciclo. Da cui: ne esiste sempre una e una sola.

3. n **nodi,** $n-1$ **lati:** Sollevando la radice, ogni nodo (tranne la radice stessa) è "tenuto su" esattamente da un filo teso dall'alto verso il basso. Se ci sono n nodi, ci sono $n-1$ nodi "tenuti su", quindi esattamente $n-1$ fili.

Il "Perché" è importante: Gli alberi introducono la dimensione logaritmica nell'informatica. Mentre le liste lineari forzano esplorazioni lunghe $O(n)$, la ramificazione degli alberi permette di immagazzinare enormi quantità di informazioni con "profondità" molto basse. Questa gerarchia è il fondamento dei file system moderni (cartelle e sottocartelle) e dei database di ricerca ad alte prestazioni.

Ripasso Visivo/Schematico:

Terminologia	**Definizione e Proprietà**
Foglie (Nodi Esterni)	Vertici senza figli (grado in uscita = 0).
Nodi Interni	Vertici con almeno un figlio.
Altezza (h)	Massima distanza (in lati) dalla radice a una foglia.
Livello Saturo	In un albero k-ario, un livello i che possiede esattamente k^i nodi.

17 Marzo 2026 — Proprietà Matematiche e Alberi Binari

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale: Albero Pieno vs Completo**[4]

[4] **Albero Pieno vs Completo:** Un albero binario *pieno* ha tutti i livelli saturi, dal primo all'ultimo. Un albero binario *completo* ha tutti i livelli saturi tranne possibilmente l'ultimo, ma i nodi dell'ultimo livello sono addensati contiguamente a partire dalla posizione più a sinistra, senza interruzioni.

- **Albero Degenere**[5]

- **La soluzione:** Il problema dell'efficienza della ricerca si traduce nella minimizzazione dell'altezza h dato un numero prefissato di vertici n. L'ottimizzazione topologica coincide con la struttura dell'albero completo. Per un albero binario completo, la sommatoria dei nodi saturi è:
$$\sum_{i=0}^{h-1} 2^i = 2^h - 1$$
L'ultimo livello h possiede almeno 1 nodo e al massimo 2^h nodi. L'equazione di bilanciamento globale è:
$$2^h \leq n \leq 2^{h+1} - 1$$
Applicando le proprietà dei logaritmi, si evince che in un albero ottimizzato l'altezza è logaritmica rispetto alla grandezza dell'input:
$$h \sim \log_2 n$$
In netta contrapposizione, il numero di possibili permutazioni strutturali di un albero degenere di n nodi si ottiene codificando la scelta (sinistra=0, destra=1) per i figli lungo $n-1$ transizioni, portando a 2^{n-1} variazioni esponenziali.

Guida di Studio Approfondita

Il Concetto Base: Immaginiamo di voler immagazzinare 1.000.000 di record. Vogliamo strutturarli in modo che, partendo dall'alto, il "viaggio" per arrivare al record più remoto sia il più breve possibile (minimizzare l'altezza). La strategia vincente è compattare il più possibile: ogni nodo deve avere due figli, saturando ogni riga prima di scendere alla successiva (Albero Completo).

Analisi Passo-Passo (Il paradosso della ricerca):

1. **Topologia degenere:** Ogni nodo ha un solo figlio. Costruisci una scala rettilinea. Hai 10^9 record? L'altezza dell'albero è un miliardo. Caso peggiore assoluto per i tempi d'attesa.

2. **Topologia ottimizzata (Completa):** Sfruttando la base binaria, il livello 0 ha 1 nodo, il livello 1 ne ha 2, il livello 2 ne ha 4, il livello 3 ne ha 8.

3. L'altezza scala con il logaritmo. Per 10^9 record (un miliardo), la formula logaritmica ci dà un'altezza di soli ≈ 30 passi.

Il "Perché" è importante: La relazione $h \sim \log_2 n$ è una delle più cruciali in tutta l'informatica strutturale. Spiega il motivo per cui l'accesso alle informazioni in un gigantesco database indicizzato avviene istantaneamente (in circa 30 operazioni) rispetto alle innumerevoli ore che ci vorrebbero se dovessimo cercare sequenzialmente riga per riga.

Ripasso Visivo/Schematico:

Tipologia di Albero	**Altezza (per n nodi)**	**Prestazione di Attraversamento**
Binario Pieno/Completo	$h \sim \log_2 n$	Eccellente. La decrescita del cammino garantisce performance massime.
Binario Degenere	$h = n - 1$	Pessima. Equivale a una ricerca sequenziale lineare su lista.

[5] **Albero Degenere:** Albero avente n nodi e un'altezza pari a $n-1$. In tale configurazione, ogni nodo interno possiede unicamente grado di uscita 1, rendendo la struttura computazionalmente isomorfa a una lista concatenata unidimensionale.

Capitolo 12

Rappresentazione di Grafi e Teoria degli Alberi

19 Marzo 2026 Componenti Connesse e Rappresentazione dei Grafi

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale: Componente Connessa**[1]
- **Rappresentazione tramite Liste di Adiacenza**[2]
- **Rappresentazione tramite Matrice di Adiacenza**[3]
- **La soluzione:** La scelta della struttura dati influisce in modo determinante sui costi computazionali, sia spaziali che temporali:
  - **Spazio Matrice di Adiacenza:** Richiede una complessità spaziale $\Theta(N^2)$. Essendo simmetrica nei grafi non orientati, si può ottimizzare salvando solo la matrice triangolare superiore, ma asintoticamente il costo rimane quadratico.
  - **Spazio Liste di Adiacenza:** Lo spazio dipende strettamente dal numero di nodi N e di lati M. La complessità spaziale è pari all'abito su misura: $\Theta(N + 2M)$ nei grafi non orientati, poiché ogni lato (x, y) appare due volte (una nella lista di x e una nella lista di y). Nei grafi orientati, lo spazio è $\Theta(N + M)$.
  - **Verifica Esistenza Lato (x, y):** Sulla Matrice di Adiacenza il test ha costo $\Theta(1)$. Sulle Liste di Adiacenza si deve scorrere la lista di un nodo alla ricerca dell'altro. Nel caso peggiore, la ricerca richiede di scandire tutti gli $N - 1$ nodi, con un costo di $O(N)$.

[1] **Componente Connessa:** Sottografo massimale in cui ogni vertice è raggiungibile dagli altri vertici appartenenti alla stessa componente. Definisce un "insieme quoziente" dove tutti i nodi al suo interno sono reciprocamente raggiungibili.

[2] **Rappresentazione tramite Liste di Adiacenza:** Struttura dati che prevede, per ogni nodo del grafo, l'uso di una lista contenente tutti i vertici a esso adiacenti. In un grafo non orientato, il lato (u, v) implica che v sia presente nella lista di u e viceversa.

[3] **Rappresentazione tramite Matrice di Adiacenza:** Matrice quadrata booleana di ordine $N \times N$, dove l'elemento (i, j) vale 1 se esiste un lato tra il vertice i e il vertice j, 0 altrimenti.

Guida di Studio Approfondita

Il Concetto Base: Immagina il grafo come una rete di città collegate da strade. Se hai un "Grafo Sparso" (poche strade, come le rotatorie di una provincia), usare una gigantesca tabella $N \times N$ (Matrice) sarebbe uno spreco, perché sarà piena di zeri. Le "Liste di Adiacenza" sono la soluzione migliore: crei una rubrica in cui ogni città elenca solo le vicine dirette. Se invece hai un "Grafo Denso" (tutti conoscono tutti), la tabella quadrata diventa utile: vai all'incrocio tra la riga della città A e la colonna della città B e capisci all'istante (tempo costante) se c'è una strada diretta.

Analisi Passo-Passo (Verifica lato con liste di adiacenza):

1. Devo verificare se esiste il lato tra il nodo 2 e il nodo 100.
2. Invece di controllare la lista del nodo 100, scelgo furbamente la lista più corta (o il nodo di grado minore) se conosciuto.
3. Se sono costretto a cercare alla cieca, apro la lista del nodo 2 e guardo ogni elemento. Se l'elemento 100 è l'ultimo o non c'è, avrò scansionato inutilmente $O(N)$ elementi sprecando tempo.

Il "Perché" è importante: In informatica non esiste una struttura dati perfetta in assoluto. La Matrice di Adiacenza garantisce un tempo di ricerca istantaneo ($\Theta(1)$) ma sacrifica pesantemente la memoria ($\Theta(N^2)$). Le Liste di Adiacenza ottimizzano la memoria limitandosi al necessario ($\Theta(N + M)$) ma penalizzano i tempi di ricerca ($O(N)$). Saper valutare se il grafo del problema è denso o sparso indirizza l'ingegnere verso l'unica implementazione idonea.

Ripasso Visivo/Schematico:

Rappresentazione	Vantaggio Principale	Svantaggio Principale	Applicazione Ideale
Matrice di Adiacenza	Test del lato $\Theta(1)$	Memoria $\Theta(N^2)$	Grafi Densi
Liste di Adiacenza	Memoria minima $\Theta(N + M)$	Ricerca del lato $O(N)$	Grafi Sparsi

19 Marzo 2026 Tassonomia e Definizione Formale degli Alberi

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale: Albero Libero**[4]
- **Albero con Radice**[5]
- **La soluzione:** La struttura viene esplorata differenziando i nodi rispetto al loro grado in uscita.
  - **Foglie (Nodi Esterni):** Nodi con grado in uscita nullo (privi di figli).
  - **Nodi Interni:** Nodi aventi almeno un figlio.

[4] **Albero Libero:** Modello topologico definito tramite tre proprietà matematicamente equivalenti: 1) Grafo non orientato, connesso e aciclico. 2) Grafo non orientato in cui per ogni coppia di vertici esiste uno e un solo cammino. 3) Grafo non orientato, connesso, avente esattamente N nodi e $N - 1$ lati.

[5] **Albero con Radice:** Un albero in cui un singolo vertice viene designato in modo arbitrario come radice. Questa operazione induce una gerarchia direzionale, permettendo la definizione dei concetti di livello (distanza dalla radice), padre, figlio, e così via.

Il numero totale di nodi dell'albero è rigorosamente dato dalla somma tra il numero delle foglie e il numero dei nodi interni. Si noti inoltre che sollevando idealmente la radice dell'albero, per il principio di connessione, gli altri $N-1$ nodi risulteranno sorretti esattamente dagli $N-1$ lati pendenti in maniera aciclica.

Guida di Studio Approfondita

Il Concetto Base: Pensa all'albero topologico come a un insieme di palline legate da fili non elastici. Essendo un grafo "connesso", nessuna pallina è isolata. Essendo "aciclico", non si formano collane chiuse. Questa combinazione fa sì che se scegli a caso una pallina qualsiasi (la *radice*) e la sollevi con due dita in aria, tutta la struttura penzolerà verso il basso. Chi sta appeso sotto un'altra pallina ne diventa il "figlio", e le ultime palline in fondo, che non hanno nient'altro appeso sotto, si chiamano "foglie".

Analisi Passo-Passo (Le 3 definizioni logiche equivalenti):

1. **Connesso e Aciclico:** Condizione base. Non ci sono pezzi staccati (connesso) e non ci sono loop (aciclico).

2. **Cammino unico tra ogni coppia:** Conseguenza diretta. Se ci fossero due cammini diversi tra la pallina A e la pallina B, per definizione si formerebbe un loop. Essendo aciclico, il cammino non può che essere unico.

3. **N palline, $N-1$ fili:** Se sollevo la radice (la pallina in mano mia non è sorretta da alcun filo dall'alto), tutte le altre $N-1$ palline sottostanti sono rette in aria da esattamente un filo proveniente dall'alto. Quindi per tenere sospese $N-1$ palline servono esattamente $N-1$ fili (lati).

Il "Perché" è importante: La molteplicità di queste definizioni non è una leziosità, ma un potente strumento dimostrativo. Negli algoritmi informatici, a seconda del teorema da analizzare, conviene appoggiarsi a una definizione piuttosto che a un'altra (es. usare "$N-1$ lati" per calcolare il consumo di memoria nei puntatori).

Ripasso Visivo/Schematico:

Terminologia Parentale	Posizione Topologica Indotta
Radice	Nodo senza alcun padre (livello 0).
Padre/Figlio	Relazione diretta (distanza pari a un lato).
Fratello	Nodi situati allo stesso livello che condividono il padre.
Antenato/Discendente	Relazione estesa su un cammino maggiore di un lato (es. nonno).

19 Marzo 2026 — Alberi k-ari, Binari e Metriche delle Altezze

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale: Albero k-ario**[6]

[6] **Albero k-ario:** Una struttura gerarchica in cui nessun nodo ha grado in uscita superiore a k, ed esiste almeno un vertice avente grado pari esattamente a k.

- **Albero con Radice Ordinato**[7]
- **Albero Binario**[8]
- **La soluzione:** La metrica dell'albero si valuta per saturazione. Il numero massimo di nodi in un livello i di un albero k-ario è k^i. Un livello è definito *saturo* se accoglie esattamente k^i nodi. L'equazione asintotica fondamentale si ricava dall'albero completo. In un albero binario completo ($k = 2$), che massimizza la compattezza dei nodi N riducendo l'altezza h, l'equazione di bilanciamento è:

$$\sum_{i=0}^{h-1} 2^i = 2^h - 1$$

L'altezza risulta logaritmica: $h \sim \log_2 N$. In contrapposizione, **Albero Degenere**[9]

Guida di Studio Approfondita

Il Concetto Base: Immagina di voler salvare 1.000 record in un albero in modo da poterli trovare il più rapidamente possibile scendendo dalla cima (radice) verso le foglie. L'altezza dell'albero determina il "tempo di ricerca". Se costruisci un *Albero Completo*, raddoppi la capacità a ogni livello: la radice (1), i figli (2), i nipoti (4), ecc. Se invece costruisci un *Albero Degenere*, metti un solo figlio sotto l'altro: costruisci un grattacielo dritto e altissimo, ovvero una semplice lista.

Analisi Passo-Passo (Ottimo vs Degenere):

1. **Albero Degenere:** Ho N nodi. L'altezza diventa spaventosa: $h = N - 1$. Quanti alberi degeneri esistono con N nodi? Visto che a ogni nodo interno devo scegliere se il figlio unico è "sinistro" (0) o "destro" (1), per $N - 1$ legami avrò 2^{N-1} alberi degeneri possibili (stringhe binarie).
2. **Albero Completo:** Saturando i livelli con le potenze di 2, se N vale 1 miliardo (10^9), il logaritmo in base 2 ci dice che l'altezza sarà poco meno di 30. Un crollo esponenziale dei tempi di attesa.

Il "Perché" è importante: Distinguere tra *Pieno* (tutti i livelli saturi) e *Completo* (tutti saturi tranne l'ultimo, che deve essere compattato tutto a sinistra) è vitale. Negli algoritmi d'implementazione strutturale, mantenere la compattezza sinistra garantisce di non creare "buchi" di memoria durante la rappresentazione in array. Se $h \sim \log_2 N$, un'operazione di visita profonda non richiederà tempi spaventosi ma rimarrà efficientissima anche per big data.

Ripasso Visivo/Schematico:

Topologia Albero (Binario)	Caratteristica Strutturale	Altezza (Complessità)
Binario Pieno	Livelli totalmente saturi (2^i nodi)	$h \sim \log_2 N$ (Ottima)
Binario Completo	Satura l'ultimo livello compattando a sinistra	$h \sim \log_2 N$ (Ottima)
Binario Degenere	Un solo figlio per nodo, isomorfo a lista	$h = N - 1$ (Pessima)

[7] **Albero con Radice Ordinato:** Un albero con radice in cui viene stabilita una relazione d'ordine totale all'interno di ogni classe di equivalenza "essere fratelli" (es. primo fratello, secondo, ecc.).

[8] **Albero Binario:** Un albero in cui ogni nodo possiede al massimo due sottoalberi disgiunti e spazialmente differenziati (figlio sinistro e figlio destro). Topologie speculari risultano non isomorfe.

[9] **Albero Degenere:** Albero in cui ogni nodo possiede esattamente un figlio (grado in uscita = 1), determinando una lista di altezza $h = N - 1$.

Capitolo 13

Proprietà degli Alberi e Loro Rappresentazioni

23 Marzo 2026 Alberi Binari Localmente Completi e Numeri di Catalano

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale: Albero Binario Localmente Completo**[1]
- **Teorema sulle foglie:** In un albero binario localmente completo, se il numero di nodi interni è n, il numero totale di foglie (nodi esterni) è rigorosamente pari a $n + 1$.
- La dimostrazione procede per induzione sul numero di nodi interni n:
  - *Caso Base ($n = 0$):* L'albero ha solo la radice, che non avendo figli è una foglia. Quindi $n = 0 \implies 0 + 1 = 1$ foglia. Verificato.
  - *Passo Induttivo:* Supponiamo l'asserto vero per alberi fino a $n - 1$ nodi interni. Un albero con n nodi interni ha una radice (che è 1 nodo interno) e due sottoalberi. Siano $k-1$ i nodi interni del sottoalbero sinistro e $n-k$ i nodi interni del sottoalbero destro. Per ipotesi induttiva, il numero di foglie del primo è $(k - 1) + 1 = k$, mentre per il secondo è $(n - k) + 1 = n - k + 1$. Il totale delle foglie dell'albero è la somma delle foglie dei due sottoalberi: $k + (n - k + 1) = n + 1$.
- **La soluzione:** Il conteggio del numero di possibili alberi binari localmente completi formati da n nodi interni si modella attraverso un'equazione di ricorrenza non lineare, la cui soluzione definisce la sequenza dei Numeri di Catalano. Sia C_n il numero di tali alberi:

$$C_0 = 1$$

$$C_n = \sum_{k=1}^{n} C_{k-1} C_{n-k}$$

La formula chiusa corrispondente per il calcolo combinatorio è:

$$C_n = \frac{1}{n+1}\binom{2n}{n}$$

[1] **Albero Binario Localmente Completo:** Un albero binario in cui ogni nodo interno possiede esattamente due figli. Qualsiasi albero binario può essere trasformato in un albero localmente completo introducendo un "nodo fittizio" che funge da figlio per tutti i nodi che non hanno grado di uscita pari a due.

Guida di Studio Approfondita

Il Concetto Base: In informatica, gli alberi binari possono essere "disordinati" (nodi con un solo figlio). Per facilitare le operazioni e i teoremi matematici, inventiamo un trucco: attacchiamo un "nodo fantasma" (fittizio) ovunque manchi un figlio. Così, l'albero diventa "localmente completo" (o ha 2 figli o non ne ha nessuno). Una volta fatto questo, emerge una magia matematica: se conti i bivi (nodi interni), i punti finali (le foglie) saranno sempre esattamente uno in più dei bivi.

Analisi Passo-Passo (L'equazione di ricorrenza di Catalano): Se vogliamo sapere in quanti modi diversi possiamo costruire un albero localmente completo con n nodi interni, dobbiamo fare una moltiplicazione di possibilità.

1. Fissiamo la radice (1 nodo). Ci restano $n-1$ nodi interni da distribuire.
2. Mettiamo $k-1$ nodi a sinistra e $n-k$ nodi a destra.
3. Il numero di alberi possibili a sinistra è C_{k-1} e a destra è C_{n-k}.
4. Per ogni singola forma dell'albero sinistro, possiamo abbinare tutte le forme possibili dell'albero destro. Ecco perché si moltiplicano: $C_{k-1} \times C_{n-k}$.
5. Si sommano i risultati per tutte le possibili divisioni (tutti i possibili valori di k).

Il "Perché" è importante: I Numeri di Catalano crescono molto velocemente, ma molto meno del numero di alberi degeneri (che crescevano come 2^{n-1}). Conoscere l'esatta cardinalità degli alberi di una certa forma è fondamentale per calcolare il caso medio degli algoritmi di ricerca.

Ripasso Visivo/Schematico:

Parametro	**Valore (Albero Localmente Completo)**
Nodi interni	n
Foglie	$n+1$
Totale Nodi	$2n+1$
Possibili forme distinte	$\frac{1}{n+1}\binom{2n}{n}$ (Numeri di Catalano)

23 Marzo 2026 Lunghezze dei Cammini e Rappresentazioni in Memoria

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale:** Valutare l'efficienza degli algoritmi di attraversamento e ricerca su strutture ad albero richiede la misurazione della **Lunghezza del Cammino Interno (LCI)**[2] e della **Lunghezza del Cammino Esterno (LCE)**[3]
- **La soluzione:** Per gli alberi binari localmente completi, esiste una relazione lineare esatta tra LCE, LCI e il numero di nodi interni n:

$$LCE = LCI + 2n$$

La dimostrazione è costruttiva:

[2]**Lunghezza del Cammino Interno (LCI):** La somma delle profondità (o livelli) di tutti i nodi interni di un albero.

[3]**Lunghezza del Cammino Esterno (LCE):** La somma delle profondità di tutte le foglie di un albero.

1. Si parte dalla sola radice ($n = 0$). LCE $= 0$, LCI $= 0$. $0 = 0 + 0$.
2. Ad ogni passo induttivo, si seleziona una foglia al livello K e la si trasforma in nodo interno (aggiungendole due figli).
3. La variazione su LCI è $+K$ (la foglia diventa nodo interno).
4. La variazione su LCE è $-K$ (perde la foglia al livello K) $+2(K + 1)$ (guadagna due nuove foglie al livello successivo). Variazione netta: $K + 2$.
5. Ad ogni inserimento di nodo interno, la differenza tra LCE e LCI aumenta esattamente di 2. Dopo n passi, la differenza totale accumulata è $2n$.

- L'implementazione in memoria della topologia ad albero si realizza storicamente con tre possibili strutture dati (array o record con puntatori):
  - **Vettore dei padri:** Il vettore mappa in posizione i l'indice del nodo padre. Risulta ottimale per la risalita (bottom-up) ma perde l'informazione di ordinamento orizzontale e rende l'attraversamento top-down oneroso ($O(n)$ per trovare un figlio).
  - **Primo figlio / Fratello successivo:** Coppia di vettori che permette di rappresentare alberi k-ari (di grado arbitrario) risolvendo il non-determinismo del numero di figli.
  - **Figlio Sinistro / Figlio Destro:** Allocazione dinamica tramite record formati da 3 campi: il dato informativo e due puntatori (`SX` e `DX`).

Guida di Studio Approfondita

Il Concetto Base: Immaginiamo di misurare la "fatica" totale per raggiungere tutti i nodi di un albero partendo dalla radice. La somma della fatica per raggiungere i nodi intermedi (LCI) e quella per raggiungere i nodi terminali (LCE) sono legate. Ogni volta che prolunghiamo l'albero "spuntando" un nodo terminale per farlo diventare un bivio con due nuove foglie, la fatica esterna cresce sempre di 2 unità in più rispetto alla fatica interna.

Analisi Passo-Passo (Le Rappresentazioni): Come salviamo fisicamente un albero nella RAM?

1. **Il vettore dei Padri:** Scriviamo un array in cui la cella 5 contiene il numero 2 (il padre del nodo 5 è il nodo 2). È comodissimo per dire "chi è il mio antenato?". Ma non sa distinguere se il 5 è il figlio sinistro o destro. E se dal nodo 2 voglio trovare i figli, devo fare una ricerca lineare scansionando tutto l'array.

2. **Primo Figlio / Fratello:** Se un nodo ha 100 figli, non posso avere record con 100 frecce. Uso due sole frecce: una punta al mio primogenito, l'altra punta a mio fratello minore. Il primogenito punterà al suo fratello minore e così via, creando una lista concatenata di fratelli.

Il "Perché" è importante: Il teorema $LCE = LCI + 2n$ è il cuore matematico che permette di dimostrare in futuro che un algoritmo di ricerca (es. ricerca binaria o ricerca in un albero) non può superare certe performance nel caso medio senza abbattere l'altezza dell'albero stesso. Scegliere la giusta rappresentazione in memoria (es. Vettore dei Padri per l'algoritmo UNION-FIND) è ciò che fa la differenza tra un algoritmo inefficiente e uno eccellente.

Ripasso Visivo/Schematico:

Struttura Dati	**Vantaggio**	**Svantaggio**
Vettore dei Padri	Risalita $O(1)$	Discesa $O(n)$, Nessun ordine
Figlio Sinistro / Destro	Discesa $O(1)$	Nessun accesso diretto al padre
Primo figlio / Fratello	Supporta grado arbitrario (k-ario)	Visita dei figli $O(k)$ (Lineare su lista fratelli)

23 Marzo 2026 Serie Geometrica Troncata e Massimizzazione dei Nodi

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale:** Calcolare il massimo numero di nodi ammissibili per un albero di grado K (albero k-ario) e altezza h. Tale configurazione si ottiene assumendo che la struttura sia un albero completo e pieno, in cui ogni livello i risulta saturo con esattamente K^i nodi.

- Il numero totale di nodi per un albero k-ario completo di altezza h è espresso dalla sommatoria:
$$N_{max} = \sum_{i=0}^{h} K^i$$

- **La soluzione:** La semplificazione di questa espressione richiede la determinazione della forma chiusa della **Serie Geometrica Troncata**[4] Sia $f_n(x) = \sum_{i=0}^{n} x^i$. Per ogni $x > 1$, moltiplicando per il fattore $(x-1)$ si ottiene:
$$\begin{aligned}(x-1)f_n(x) &= x\sum_{i=0}^{n} x^i - 1\sum_{i=0}^{n} x^i \\ &= \sum_{i=0}^{n} x^{i+1} - \sum_{i=0}^{n} x^i \\ &= (x^1 + x^2 + \cdots + x^{n+1}) - (x^0 + x^1 + \cdots + x^n)\end{aligned}$$
I termini intermedi si annullano a vicenda (elisione telescopica), lasciando unicamente $x^{n+1} - x^0$, da cui:
$$f_n(x) = \frac{x^{n+1} - 1}{x - 1}$$
Di conseguenza, il massimo numero di nodi in un albero k-ario di altezza h è rigorosamente calcolabile come:
$$N_{max} = \frac{K^{h+1} - 1}{K - 1}$$

Guida di Studio Approfondita

Il Concetto Base: Immagina un albero di grado 3 (ogni nodo ha 3 figli) alto 4 livelli. Quanti nodi ci sono in tutto? Al livello 0 c'è 1 nodo. Al livello 1 ce ne sono 3. Al livello 2 ce ne sono 9. Al livello 3 ce ne sono 27, e al livello 4 ce ne sono 81. Sommarli a mano $(1 + 3 + 9 + 27 + 81)$ è facile se l'albero è piccolo, ma se l'altezza fosse 100? La matematica ci fornisce una formula "istantanea" (forma chiusa) sfruttando la serie geometrica troncata.

Analisi Passo-Passo (Il trucco della moltiplicazione):

1. Considera la somma $S = 1 + x + x^2 + x^3$.
2. Moltiplichiamo tutto per x: $x \cdot S = x + x^2 + x^3 + x^4$.

[4] **Serie Geometrica Troncata:** Un polinomio in cui sono presenti tutti i monomi dalla potenza 0 alla potenza n con coefficiente unitario.

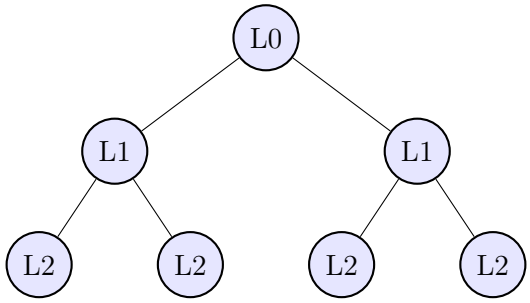
3. Sottraiamo la prima dalla seconda: $x \cdot S - S = (x + x^2 + x^3 + x^4) - (1 + x + x^2 + x^3)$.
4. Tutti i termini intermedi si cancellano. Rimane solo: $S(x - 1) = x^4 - 1$.
5. Dividendo per $(x - 1)$ isoliamo la nostra somma S.

Il "Perché" è importante: Grazie a questa equazione con un semplice elevamento a potenza, una sottrazione e una divisione ($\Theta(1)$ se le dimensioni numeriche lo consentono, in costo uniforme) possiamo determinare istantaneamente i limiti architetturali di un database o di una struttura dati complessa (es. B-Tree). Se usiamo un albero ternario ($K = 3$) alto h, il limite massimo è $\frac{3^{h+1}-1}{2}$.

Ripasso Visivo/Schematico:

Grado dell'Albero (K)	**Formula per i nodi totali (h)**	**Esempio Altezza $h = 3$**
Binario ($K = 2$)	$\frac{2^{h+1}-1}{2-1} = 2^{h+1} - 1$	$2^4 - 1 = 15$ nodi
Ternario ($K = 3$)	$\frac{3^{h+1}-1}{2}$	$\frac{81-1}{2} = 40$ nodi
Quaternario ($K = 4$)	$\frac{4^{h+1}-1}{3}$	$\frac{256-1}{3} = 85$ nodi

Esempio Visivo: Albero Binario ($K = 2$) di Altezza $h = 2$



Verifica Matematica:
Applicando la formula per $K = 2$ e $h = 2$:

$$N_{max} = \frac{2^{2+1} - 1}{2 - 1} = \frac{2^3 - 1}{1} = 8 - 1 = 7 \text{ nodi totali.}$$

Come si può contare dal grafico, i nodi sono esattamente 7 (1 radice, 2 figli, 4 nipoti).

Capitolo 14

Visita di Grafi e Alberi

24 Marzo 2026 Attraversamento in Profondità (DFS) nei Grafi

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale: Visita in Profondità (DFS)**[1]
- L'assunzione strutturale prevede l'utilizzo di un vettore booleano (o binario) `nuovo/vecchio` per tracciare lo stato dei vertici, evitando cicli infiniti.
- **La soluzione:** L'implementazione iterativa si appoggia su una struttura dati Pila (Stack). Il nodo sorgente viene pushato nello Stack. Finché lo Stack non è vuoto, si estrae un nodo, lo si visita (marcandolo come vecchio) e si pushano nello Stack i suoi nodi adiacenti ancora marcati come "nuovi".
- La complessità asintotica temporale e spaziale è la seguente:

$$S(n) = \Theta(n)$$

$$T(n, m) = \Theta(n + m)$$

  Dove n è il numero di vertici e m il numero di lati. Ogni nodo viene inserito ed estratto dalla Pila al massimo una volta, e le liste di adiacenza vengono scansionate una sola volta.

Guida di Studio Approfondita

Il Concetto Base: Immagina di esplorare un labirinto. Invece di guardare tutte le strade vicine (come nella visita in ampiezza), prendi una strada e vai dritto finché non sbatti contro un muro o arrivi in un posto che hai già visitato. A quel punto, fai un passo indietro (backtracking) e provi un'altra deviazione. Per non perderti usi la strategia di Pollicino: lo Stack (la Pila) è il tuo percorso di "briciole di pane" per ricordarti come tornare indietro.

Analisi Passo-Passo:

[1] **Visita in Profondità (DFS):** Strategia di attraversamento di un grafo in cui ci si allontana il più possibile dal nodo sorgente prolungando il cammino lungo nodi non ancora visitati. Raggiunto un nodo senza adiacenti "nuovi" (vicolo cieco), si applica il backtracking, retrocedendo al nodo precedente per esplorare percorsi alternativi.

1. **Avanzamento:** Ti trovi nel nodo A. Guardi i vicini. B non è mai stato visitato. Metti A nello Stack per ricordarti che devi tornarci, e ti sposti in B.
2. **Backtracking (Pop):** Arrivi in un nodo D. Guardi i suoi vicini e scopri che sono tutti "vecchi" (già esplorati). D non ha più nulla da offrirti. Fai un `Pop` dallo Stack per riemergere nel nodo precedente (es. C) e guardi se C ha altri vicini inesplorati.
3. **Terminazione:** L'algoritmo finisce quando torni al punto di partenza e svuoti completamente lo Stack, scoprendo che non ci sono più strade nuove da prendere.

Il "Perché" è importante: La DFS è la base algoritmica per risolvere problemi complessi su percorsi e connettività, come trovare l'uscita da un labirinto, ordinare topologicamente una serie di task con dipendenze, o individuare cicli all'interno di una rete (es. deadlock in un sistema operativo).

Ripasso Visivo/Schematico:

Parametro Stack (Pila)	**Grafo: Caso Peggiore**	**Grafo: Caso Migliore**
Altezza Max Stack (h)	$h = n$ (Grafo lineare / Lista)	$h = 1$ (Grafo a stella)
Comportamento	Simmetrico e opposto alla visita BFS	Simmetrico e opposto alla BFS

24 Marzo 2026 Attraversamento degli Alberi: Preorder, Inorder, Postorder

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale:** Dato un albero con radice ordinato, determinare sequenze di attraversamento che visitino tutti i nodi una e una sola volta. La tassonomia delle visite deriva dalla posizione temporale in cui viene visitata la radice rispetto ai suoi sottoalberi.
- **Visita in Ordine Anticipato (Preorder)**[2]
- **Visita in Ordine Posticipato (Postorder)**[3]
- **Visita in Ordine Simmetrico (Inorder)**[4]
- **La soluzione:** La formulazione base degli algoritmi è intrinsecamente ricorsiva. Il tempo di calcolo è dominato dal numero di nodi n, in quanto ciascun nodo subisce un numero costante di operazioni elementari (test, chiamata a funzione).

  $$T(n) = \Theta(n)$$

  La complessità spaziale $S(n)$ dipende invece dall'altezza dell'albero h, a causa del Record di Attivazione impilato sullo Stack di sistema durante le chiamate ricorsive:

  $$S(n) = O(h)$$

[2] **Visita in Ordine Anticipato (Preorder):** Visita prima la radice, poi ricorsivamente i sottoalberi procedendo da sinistra verso destra.

[3] **Visita in Ordine Posticipato (Postorder):** Visita prima i sottoalberi ricorsivamente da sinistra a destra, e infine la radice.

[4] **Visita in Ordine Simmetrico (Inorder):** Valida per alberi binari. Visita prima il sottoalbero sinistro, poi la radice, infine il sottoalbero destro.

Ne consegue che, nel caso peggiore (albero degenere), lo spazio è $\Theta(n)$, mentre per un albero bilanciato (es. albero binario completo) lo spazio si comprime a $\Theta(\log n)$.

Guida di Studio Approfondita e Semplificata

Il Concetto Base: Immagina un albero genealogico. Se vuoi elencare le persone: - Se usi il **Preorder**, citi prima il nonno, poi il padre, poi il figlio maggiore. Il "capo" (radice) ha la precedenza assoluta. - Se usi il **Postorder**, citi prima l'ultimo discendente, poi risali al padre, e infine al nonno. Prima i "sottoposti", alla fine il "capo". - Se usi l'**Inorder** (su un albero binario), citi prima il ramo sinistro della famiglia, poi il capo in mezzo, poi il ramo destro.
Analisi Passo-Passo (L'albero stampato): Esiste anche l'**Attraversamento per Livelli** (Level-order). Invece di usare una Pila, questo usa una **Coda (Queue)**.

1. Legge il livello 0 (la radice). Mette in coda i figli.
2. Estrae dalla coda i nodi del livello 1 da sinistra a destra, mettendone in coda i relativi figli (livello 2).
3. Procede a strati, come se affettasse l'albero orizzontalmente. È la trasposizione diretta della Visita in Ampiezza (BFS) vista per i grafi, semplificata dall'assenza di cicli.

Il "Perché" è importante: La scelta della visita cambia radicalmente l'uso dei dati. L'Inorder su un albero di ricerca binaria stamperà i dati perfettamente in ordine crescente. Il Postorder è indispensabile per cancellare un albero dalla memoria: devi prima liberare la memoria occupata dai figli, altrimenti cancellando la radice perderesti i puntatori per raggiungerli!
Ripasso Visivo/Schematico:

Strategia	Ordine di Visita Strutturale	Struttura Dati Sottostante
Preorder	Radice → Sinistra → Destra	Pila (Ricorsione di Sistema)
Inorder	Sinistra → Radice → Destra	Pila (Ricorsione di Sistema)
Postorder	Sinistra → Destra → Radice	Pila (Ricorsione di Sistema)
Per Livelli	Radice, poi Livello 1, poi Livello 2...	Coda (Iterativo)

24 Marzo 2026 — Eliminazione della Ricorsione e Ricorsione Terminale

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale: Ricorsione Terminale (Tail Recursion)**[5]
- I compilatori moderni applicano un'ottimizzazione automatica (Tail Call Optimization) su queste procedure. Se una ricorsione non è terminale, ogni invocazione obbliga il sistema operativo ad allocare un nuovo Record di Attivazione (con parametri locali, puntatori di ritorno, ecc.) sullo Stack di sistema.

[5] **Ricorsione Terminale (Tail Recursion):** Particolare tipologia di funzione ricorsiva in cui la chiamata a se stessa rappresenta rigorosamente l'ultima istruzione logica da eseguire nel flusso di controllo, non essendoci operazioni in sospeso da risolvere al rientro dalla chiamata.

- **La soluzione:** L'eliminazione della ricorsione terminale prevede la trasmutazione del codice in forma iterativa. La trasformazione sostituisce l'istruzione di chiamata ricorsiva `f(g(x))` con l'aggiornamento diretto del parametro locale `x = g(x)`, avvolgendo il corpo della funzione in un costrutto iterativo `while(!condizione_di_arresto)`.

$$\text{Spazio Ricorsivo: } O(n) \implies \text{Spazio Iterativo: } O(1)$$

- Per la ricorsione non terminale (es. l'attraversamento Inorder o Postorder), la de-ricorsivazione impone la creazione esplicita di uno Stack a livello applicativo (heap memory) contenente solo i riferimenti necessari (puntatori ai nodi), sgravando lo Stack di sistema dal peso dei metadati dei Record di Attivazione.

Guida di Studio Approfondita e Semplificata

Il Concetto Base: Immagina di dover delegare un compito. Nella ricorsione normale, tu deleghi una parte del lavoro a un tuo clone, ma poi ti "congel" (occupando spazio nella stanza) aspettando che lui finisca per poter mettere la firma finale sul documento. Nella **Ricorsione Terminale**, tu deleghi l'intero lavoro al tuo clone e, visto che non devi fare più nulla dopo di lui, te ne vai a casa. Il computer (compilatore) si accorge di questo e invece di creare un clone, ti dice: "Senti, cambia solo il foglio sulla tua scrivania e ricomincia a lavorare tu, senza chiamare nessun altro".

Analisi Passo-Passo (Il problema del Postorder iterativo): Trasformare il Preorder o Inorder in iterativo è intuitivo: pushi i nodi in una Pila e li estrai. Il Postorder iterativo è infido.

1. **L'ambiguità:** Estraggo un nodo dalla Pila. Lo devo visitare (stampare)? O devo usarlo per andare a visitare i suoi figli?

2. **L'etichettatura:** Per risolvere il dilemma, inserisco nello Stack non solo il nodo, ma un `booleano`.

3. `True (Prenotato per espansione):` Significa "Ho trovato questo nodo, ma devo ancora esplorare i suoi figli". Lo reinserisco con tag `False` e pusho i suoi figli con tag `True`.

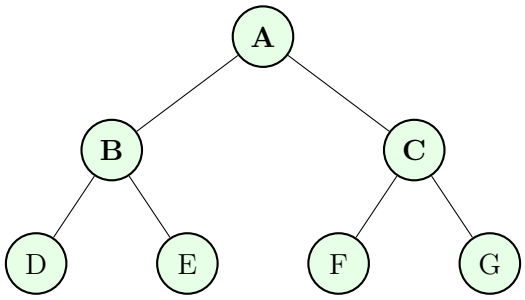
4. `False (Pronto per la visita):` Significa "I miei figli sono già stati elaborati, posso finalmente visitare me stesso".

Il "Perché" è importante: La ricorsione è matematicamente elegante ma ingegneristicamente pericolosa. Un albero "degenere" (es. una lista concatenata di 1 milione di elementi) causerebbe 1 milione di chiamate ricorsive sovrapposte. Risultato? `StackOverflowError` e crash del programma. Convertire l'algoritmo in iterativo (anche gestendo uno Stack applicativo leggero) garantisce la robustezza e la stabilità del software in produzione.

Ripasso Visivo/Schematico:

Approccio	**Vantaggi**	**Svantaggi (Rischi)**
Codice Ricorsivo	Estremamente compatto, leggibile, elegante.	Rischio `StackOverflow`, elevato memory footprint.
Iterativo Terminale	Memoria $O(1)$, veloce (evita setup funzioni).	Applicabile solo se la chiamata è in ultima posizione.
Iterativo con Stack	Memoria heap ottimizzata, robusto ai limiti.	Codice verboso, gestione manuale dei puntatori.

Esempio Visivo: Strategie di Attraversamento di un Albero Binario



Sequenze di Attraversamento Risultanti:

- **Preorder (Radice, Sinistra, Destra):**
  A → B → D → E → C → F → G
- **Inorder (Sinistra, Radice, Destra):**
  D → B → E → A → F → C → G
- **Postorder (Sinistra, Destra, Radice):**
  D → E → B → F → G → C → A

Capitolo 15

Eliminazione della Ricorsione e Introduzione all'Ordinamento

30 Marzo 2026 Attraversamento Iterativo degli Alberi: Preorder e Inorder

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale:** La dericorsivazione delle visite su alberi binari richiede la simulazione esplicita del Record di Attivazione mediante l'impiego di una struttura dati ausiliaria.
- **Visita Preorder Iterativa**[1]
- **La soluzione per il Preorder:** Inizializzato lo stack con la radice, l'algoritmo procede estraendo un nodo, visitandolo e inserendo nello stack i suoi figli. Per rispettare l'ordine topologico (Sinistra → Destra) sotto una politica LIFO, i figli devono essere inseriti nello stack in ordine inverso: prima si effettua la `push` del figlio destro, poi la `push` del figlio sinistro.
- **Visita Inorder Iterativa**[2]
- **La soluzione per l'Inorder:** L'algoritmo prevede un ciclo interno che, a partire da un nodo, scende ininterrottamente lungo il ramo sinistro eseguendo `push` di ogni nodo incontrato, finché non si incontra un riferimento nullo. A questo punto, il nodo in cima allo stack è un nodo privo di figlio sinistro: lo si estrae, lo si visita e si assegna al puntatore di scorrimento il riferimento al suo figlio destro (il quale innescherà una nuova discesa a sinistra al ciclo successivo).
- La complessità spaziale per entrambe le visite non è determinata dai Record di Attivazione di sistema, ma dalla dimensione dello stack allocato nell'heap, che nel caso peggiore (albero degenere) è:
$$S(n) = \Theta(n)$$

[1] **Visita Preorder Iterativa:** Simulazione della visita in ordine anticipato tramite una struttura Pila (Stack). Il nodo estratto viene immediatamente visitato.

[2] **Visita Inorder Iterativa:** Simulazione dell'ordine simmetrico. A differenza del Preorder, l'estrazione di un nodo dallo stack non ne implica l'immediata visita, in quanto occorre prima esaurire il sottoalbero sinistro.

Guida di Studio Approfondita

Il Concetto Base: Trasformare una funzione ricorsiva in iterativa significa "fare a mano" quello che il sistema operativo fa in automatico. Poiché il sistema usa uno Stack per ricordarsi dove tornare, noi creiamo uno Stack nostro. Nel **Preorder**, la logica è immediata: "Visito me stesso, poi mi segno su un foglio che dovrò visitare i miei figli". Visto che lo Stack estrae l'ultima cosa inserita (LIFO), per visitare prima il figlio sinistro devo inserire nello Stack prima il destro (che finirà sul fondo) e poi il sinistro (che resterà in cima, pronto per essere estratto al giro successivo).

Analisi Passo-Passo (Inorder): L'**Inorder** è più insidioso. Non posso estrarre la radice e stamparla subito.

1. **Discesa a sinistra:** Parto dalla radice e vado sempre a sinistra, impilando tutti i nodi che trovo (Radice, Figlio Sx, Nipote Sx...) finché non sbatto contro il vuoto.

2. **Visita e Deviazione:** Il nodo in cima allo Stack è sicuramente quello più a sinistra di tutti. Lo estraggo e lo stampo. A questo punto, faccio un passo a destra. Se c'è un figlio destro, riapplico la regola della discesa a sinistra partendo da lui.

Il "Perché" è importante: Perché faticare per scrivere una versione iterativa, più lunga e meno elegante di quella ricorsiva? Per risparmiare memoria fisica. Nello stack di sistema, ogni chiamata ricorsiva alloca variabili locali, indirizzi di ritorno e metadati. Nel nostro Stack iterativo, salviamo *solo* un minuscolo indirizzo di memoria (il puntatore al nodo). Questo previene i disastrosi errori di `StackOverflow`.

Ripasso Visivo/Schematico:

Strategia	**Ordine di inserimento nello Stack**	**Condizione di visita**
Preorder	1. Destro, 2. Sinistro.	Subito dopo la `pop`.
Inorder	Discesa continua a sinistra.	Quando l'estrazione non ha più figli sinistri in sospeso.

30 Marzo 2026 — Visita Iterativa in Postorder: Gestione delle Ambiguità

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale:** Sviluppare una versione iterativa pura per l'attraversamento in ordine posticipato (Postorder). Il problema ingegneristico sorge dall'ambiguità semantica: l'estrazione di un nodo dallo stack non fornisce sufficienti informazioni sul suo stato computazionale (non si sa se il sottoalbero destro è già stato esplorato o meno).

- **Stack Paralleli (o Stack di Record)**[3]

- **La soluzione:** Utilizzare il flag per discriminare le azioni.

[3] **Stack Paralleli (o Stack di Record):** Struttura dati estesa in cui ogni riferimento al nodo inserito nella pila viene accoppiato a un flag booleano che denota la condizione logica della visita (`true` o `false`).

- **Flag True (Prenotazione di espansione):** Il nodo viene estratto, ma non può essere visitato perché i suoi sottoalberi devono ancora essere elaborati. L'algoritmo re-immette il nodo nello stack con flag `false`, dopodiché esegue la `push` del figlio destro con flag `true`, seguita dalla `push` del figlio sinistro con flag `true`.
- **Flag False (Pronto per la visita):** I sottoalberi del nodo sono stati interamente elaborati. L'algoritmo procede unicamente alla visita del nodo (es. stampa).

- L'inserimento iniziale nello stack avviene caricando la radice con flag `true`. La computazione si arresta quando lo stack si svuota. La complessità temporale rimane vincolata a $\Theta(n)$, garantendo prestazioni asintotiche equivalenti alla formulazione ricorsiva.

Guida di Studio Approfondita

Il Concetto Base: Scrivere il Postorder iterativo è la vera sfida delle visite. Se impili i nodi normalmente, quando fai la 'pop' e ti trovi in mano il nodo C, il computer si blocca in un'incertezza logica: "Ho estratto C per la prima volta e devo andare a esplorare i suoi figli? O sto estraendo C per l'ultima volta tornando dai suoi figli, e quindi devo finalmente stamparlo?". La soluzione è attaccare un post-it (un booleano) a ogni nodo che infiliamo nello Stack.

Analisi Passo-Passo (Il trucco del Post-It):

1. **Post-It TRUE (Da smontare):** Estraggo un nodo (es. A) con `TRUE`. Capisco che è la prima volta che lo incontro. Lo rimetto subito dentro lo stack cambiando il post-it in `FALSE` (che significa "quando mi ripeschi, stampami"). Poi gli piazzo sopra i due figli, prima il destro poi il sinistro, entrambi con post-it `TRUE` (anche loro dovranno essere smontati).

2. **Post-It FALSE (Da stampare):** Estraggo un nodo con `FALSE`. Questo accadrà solo quando tutti i nodi che gli erano stati messi sopra (i suoi figli e nipoti) saranno stati estratti e risolti. A questo punto, posso stamparlo tranquillamente.

Il "Perché" è importante: Questa astrazione dimostra un principio teorico cardine: lo Stack di sistema che gestisce la ricorsione salva moltissime informazioni di stato. Per simulare la ricorsione più difficile (il Postorder non ha ricorsione in coda, ne ha due pendenti) non basta salvare i dati, serve salvare lo "stato di avanzamento" dell'algoritmo. L'aggiunta del flag booleano è l'implementazione minima indispensabile dello stato di sistema.

Ripasso Visivo/Schematico:

Etichetta Estratta	Significato	Azione nello Stack
`TRUE`	Albero da esplorare	1. Push(Nodo, `FALSE`) 2. Push(Figlio DX, `TRUE`) 3. Push(Figlio SX, `TRUE`)
`FALSE`	Sottoalberi completati	Visita il nodo. Nessuna operazione di Push.

30 Marzo 2026 — Introduzione al Problema dell'Ordinamento (Sorting)

Output 1: Diario Generale di Ripasso (Formato Accademico)
Struttura di un problema:

- **Partiamo da una definizione formale: Problema dell'Ordinamento**[4]
- **Tassonomia degli algoritmi di ordinamento:**
  1. **Collocazione in memoria:** *Ordinamento Interno* (l'intera sequenza è allocabile simultaneamente in RAM, accesso uniforme $\Theta(1)$) vs *Ordinamento Esterno* (i dati risiedono su memorie di massa, l'I/O determina il costo primario. Trattato in letteratura ma non approfondito qui).
  2. **Azione sui dati:** *Metodi basati su Confronti e Scambi* (i dati sono entità atomiche, si valuta la relazione d'ordine globale. Limite asintotico inferiore matematico $\Omega(n \log n)$) vs *Metodi Digitali* (si esplora la rappresentazione binaria interna della chiave).
  3. **Stabilità logica: Stabilità**[5]
  4. **Adattività: Adattivo**[6]

Guida di Studio Approfondita

Il Concetto Base: Ordinare dei dati è l'operazione più frequente in informatica (indicizzare un database, stilare una classifica, ricercare rapidamente). Il problema matematico è banale (mettere in fila dal più piccolo al più grande), ma le ripercussioni ingegneristiche sono sterminate. Per studiarli, dividiamo gli algoritmi in "famiglie" a seconda di come lavorano.

Analisi Passo-Passo (Le Proprietà Fondamentali):

1. **Interno vs Esterno:** Se vuoi ordinare un file da 1 TeraByte su un computer con 8 GigaByte di RAM, non puoi caricare tutto in memoria. L'algoritmo *Interno* fallisce (esaurimento memoria). Servono algoritmi *Esterni* che caricano blocchi, li ordinano e li riversano su disco.

2. **Confronto vs Digitale:** Ordinare per confronto è come mettere in fila le persone facendole mettere spalla a spalla per vedere chi è più alto (l'entità è considerata intera). I metodi digitali guardano "dentro" al numero, analizzando cifra per cifra (es. prima ordino per le unità, poi per le decine).

3. **Stabilità (Esempio pratico):** Immagina di avere una lista di studenti già in ordine alfabetico. Decidi di ordinarla per Voto di Laurea. Due studenti (Rossi e Verdi) hanno preso 110. In che ordine appariranno? Se l'algoritmo è *stabile*, Rossi rimarrà prima di Verdi (preserva l'alfabetico a parità di voto). Se è *instabile*, l'algoritmo li mescolerà a caso, distruggendo il lavoro precedente.

Il "Perché" è importante: Non esiste "l'algoritmo di ordinamento perfetto" in senso assoluto. Se hai dati su nastro magnetico, ti serve un ordinamento Esterno. Se hai bisogno di fare ordinamenti successivi su più chiavi (es. ordina per Nome e poi per Cognome), sei obbligato a scegliere un algoritmo Stabile. Se il vettore in ingresso è "quasi" già tutto ordinato e vuoi ottimizzare le tempistiche, sceglierai un algoritmo Adattivo (che si accorge del poco disordine e termina in tempo lineare). Conoscere le etichette permette all'ingegnere di selezionare lo strumento adatto.

Ripasso Visivo/Schematico:

[4]**Problema dell'Ordinamento:** Dato un insieme base U dotato di una relazione d'ordine totale $\leq$, e un vettore in ingresso $A = (A, \ldots, A[n])$ con $A[i] \in U$, generare una sequenza in uscita B tale che B sia una permutazione di A e valga $B[i] \leq B[i+1]$ per ogni $i \in \{1, \ldots, n-1\}$.

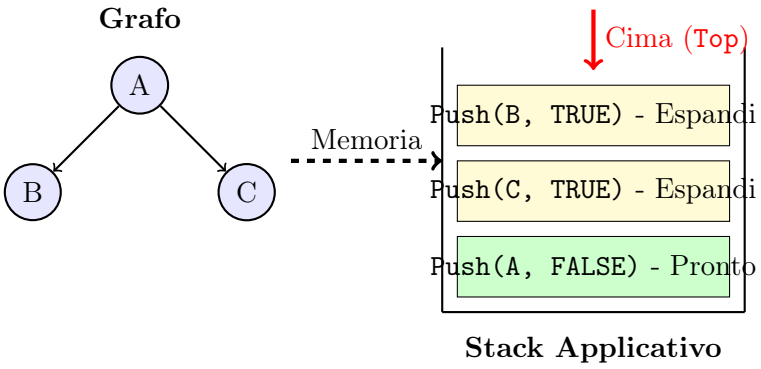
[5]**Stabilità:** Un algoritmo di ordinamento è stabile se, a parità di chiave (elementi uguali), l'algoritmo preserva rigorosamente l'ordine relativo preesistente che gli elementi avevano nella sequenza di input.

[6]**Adattivo:** La complessità computazionale (numero di confronti) varia in funzione del grado di disordine preesistente nei dati. L'algoritmo esegue permutazioni dei test a runtime.

Classificazione	Opzione A	Opzione B
Gestione Memoria	Interno (Tutto in RAM)	Esterno (Accesso a Memoria di Massa)
Tipologia Scansione	Confronti/Scambi (Dato Atomico)	Digitale (Manipolazione Bit)
Gestione Valori Uguali	Stabile (Preserva l'ordine)	Instabile (Ordine relativo alterato)
Sensibilità al Disordine	Adattivo (Dipende dall'Input)	Non Adattivo (Test precablati)

Esempio Visivo: Dinamica dello Stack nel Postorder Iterativo

Il seguente schema illustra l'istante in cui la radice **A** (estratta con flag `TRUE`) viene spacchettata. Al suo posto, vengono inseriti nello Stack, dall'alto verso il basso: il figlio sinistro da esplorare, il figlio destro da esplorare, e la radice stessa marcata come pronta.



Analisi dello Stato Logico: Al ciclo successivo, il comando `Pop()` estrarrà l'elemento `(B, TRUE)`. La macchina vedrà il flag `TRUE`, eviterà di stamparlo e procederà a spacchettare i figli di B nello Stack applicando la medesima logica, garantendo una discesa controllata senza ricorsione di sistema.